Progettazione e implementazione di uno strumento di simulazione per le Comunità Energetiche Rinnovabili: dai flussi energetici alla valutazione economica

Tesi di Laurea Magistrale

Corso di Laurea Magistrale in Data Science and Business Analytics

Candidato: FILIP JUREN
Relatore: Luigi De Paoli

Anno Accademico 2024/25

Indice

1 Inquadramento e obiettivi del simulatore **1**

1.1 Contesto: comunità energetiche rinnovabili e autoconsumo diffuso . . 1

1.2 Obiettivi della tesi e perimetro del lavoro 2

1.3 Assunzioni generali e convenzioni 4

1.3.1 Orchestrazione della pipeline e responsabilità dei moduli . . . 4

1.3.2 Convenzione temporale e risoluzioni 5

1.3.3 Sessioni, run, riproducibilità e tracciabilità 5

2 Architettura del simulatore **7**

2.1 Organizzazione del software 7

2.2 Persistenza, sessioni, scenari e run 9

2.3 Descrizione del workflow 10

2.3.1 Risoluzione dei percorsi e inizializzazione della sessione 10

2.3.2 Home e meteo 10

2.3.3 Produttori 10

2.3.4 Consumatori 11

2.3.5 Ponte di integrazione e materializzazione degli input 11

2.3.6 Bilanciamento energetico 12

2.3.7 Valutazione economica 12

2.3.8 Ottimizzazione 13

2.4 Convenzioni e contratti dati 13

2.5 Rimandi ai capitoli successivi 14

3 Condizioni generali: configurazione globale e componente climatica **15**

3.1 Sessione di simulazione: metadati e persistenza 16

3.2 Scelta dell'intervallo temporale e convenzione UTC 16

3.3 Selezione della località tramite Geocoding API 17

3.4 Acquisizione del meteo orario via Open-Meteo 17

3.5 Output meteo di sessione e feature engineering 19

3.6 Organizzazione dei file su disco (audit della sessione) 19
3.7 Visualizzazione e ispezione dei dati 20

4 Produzione energetica 21
4.1 Scopo del capitolo e inquadramento nella pipeline 21
4.2 Sessione, input comuni e artefatti su disco 22
4.2.1 File rilevanti per la produzione 22
4.2.2 Responsabilità dei moduli: UI vs core 22
4.3 Workflow della pagina `Producers` e persistenza della configurazione . 23
4.3.1 Creazione/modifica dei produttori (CRUD) 23
4.3.2 Definizione delle aree FV e convenzioni geometriche 24
4.4 Produzione fotovoltaica 24
4.4.1 Stima della potenza installabile: packing nel Local Area Frame 24
4.4.2 Interrogazione PVGIS e convenzione degli angoli 25
4.4.3 Caching della produzione FV 25
4.4.4 Input del modello fotovoltaico 25
4.5 Produzione eolica 26
4.5.1 Pipeline tecnica: meteo → velocità a quota mozzo → power curve 26
4.5.2 Libreria turbine e override della curva 27
4.5.3 Caching della produzione eolica 27
4.5.4 Input del modello eolico 28
4.6 Output prodotti e utilizzo nei moduli successivi 28
4.7 Verifiche qualitative e limiti del modello 28
4.8 Figure del capitolo 29

5 Domanda elettrica: modellazione bottom-up dei consumi 31
5.1 Modello e integrazione nella pipeline 31
5.1.1 Inquadramento e requisiti del sottosistema domanda 31
5.1.2 Razionale modellistico: perché questa implementazione 32
5.1.3 Struttura del workflow e artefatti prodotti 32
5.1.4 File generati e convenzioni di output 32
5.2 Moduli di generazione della domanda 33
5.2.1 Carichi base: always-on, standby e cicli (Baseload) 33
5.2.2 Occupancy-driven: presenza, attività e illuminazione 34
5.2.3 Cucina: modello event-driven basato su pasti 35
5.2.4 Lavanderia: cicli discreti con vincoli di scheduling 36
5.2.5 Clima: modello termo-elettrico compatto e dipendenza dal meteo 37

5.3 Aggregazione, riscalatura e interpretazione dei risultati 38
5.3.1 Aggregazione del profilo totale 38
5.3.2 Riscalatura su target energetico annuo 38
5.4 Esempi di profili e discussione qualitativa 38

6 Bilanciamento energetico: implementazione, flussi e energia condivisa 40
6.1 Ruolo del sottosistema e contratto dati nella pipeline 40
6.2 Scenario workspace e persistenza degli input 41
6.2.1 Struttura su filesystem per scenario e per membro 41
6.2.2 Modello dati dello scenario e migrazione legacy 41
6.3 Coerenza temporale e data quality 42
6.3.1 Normalizzazione timezone e convenzione UTC 42
6.3.2 Definizione del periodo e costruzione degli indici attesi (T15, TH) 42
6.3.3 Validazioni e regole conservative sui dati mancanti 42
6.4 Parsing e costruzione delle serie di consumo e produzione 43
6.4.1 Consumi a 15 minuti: formato, controlli e clipping 43
6.4.2 Produzione oraria: modalità Totale vs Aree e riallineamento su T15 43
6.5 Bilancio energetico del membro (senza accumulo) 43
6.5.1 Notazione, unità e conversione kW→kWh 43
6.5.2 Definizione di autoconsumo, prelievo e immissione 44
6.5.3 Aggregazione oraria dei flussi 44
6.6 Estensione con accumulo elettrochimico (BESS) in logica RSE-like . . 44
6.6.1 Parametri, vincoli SOC e efficienze (half-cycle) 44
6.6.2 Step di simulazione a 15 minuti e aggiornamento del SOC . . 45
6.6.3 Discussione del livello di astrazione (energy-based) e implicazioni 45
6.7 Aggregazione di comunità ed energia condivisa 46
6.7.1 Somma dei flussi e definizione dell'energia condivisa oraria . . 46
6.7.2 Residui verso rete (import/export) e controlli di coerenza . . . 46
6.8 Output del bilanciamento e versionamento dei run 46
6.8.1 Schema degli output (CER, membri long, summary, metadati) 46
6.8.2 Registro append-only, run attivo e snapshot di configurazione 47
6.9 Confronto mirato con CACER_Simulator (RSE) 47
6.10 Limiti noti ed estensioni 47

7 Valutazione economica dei progetti CER e ottimizzazione del BESS 49
7.1 Dati in ingresso e collegamento al run energetico 49
7.2 Assunzioni economiche e profili di prezzo/incentivo 50
7.2.1 Profili temporali: convenzioni, unità e validazione 50
7.2.2 Prezzo zonale orario: definizione e modalità di acquisizione . . 51
7.2.3 Prezzo di acquisto dell'energia 52
7.2.4 Prezzo di vendita dell'energia 52
7.2.5 Profilo di incentivo TIP 53
7.2.6 Profilo di restituzione componenti TIAD 54
7.2.7 Detrazione fiscale (Bonus ristrutturazioni): parametrizzazione 54
7.2.8 Osservazioni operative e limiti 55
7.3 Costi di acquisto, ricavi da vendita e incentivi 55
7.4 Allocazione degli incentivi ai membri 56
7.5 Bilancio economico annuale e modello DCF 57
7.6 Output economici e tracciabilità 59
7.7 Ottimizzazione del dimensionamento BESS in funzione dell'NPV . . . 59
7.7.1 Problema di allocazione e ipotesi 60
7.7.2 Algoritmo greedy a incremento di capacità 60
7.7.3 Criteri di arresto e output 61
7.7.4 Considerazioni 61

8 Caso studio: costruzione del dataset e impostazione dello scenario 62
8.1 Raccolta dati primari: interviste in presenza e rilievo impianti 63
8.2 Stima della produzione fotovoltaica: PVGIS e configurazione reale dei tetti 63
8.3 Curve di consumo ARERA come benchmark: motivazione e fonte . . 64
8.4 Dal dato ARERA *mese* × *tipo giorno* a una serie annuale compatibile col simulatore 65
8.4.1 Pulizia e ristrutturazione dei file (filter + pivot) 65
8.4.2 Costruzione del calendario annuale e mapping del *tipo giorno* . 66
8.4.3 Join e ricostruzione della serie oraria 66
8.4.4 Riscalatura al consumo annuo dell'utenza: da profilo "shape" a profilo "utente" 66
8.4.5 Conversione $T_H \to T_{15}$ per confronto diretto 67
8.5 Output atteso della fase dati e integrazione nello scenario 67
8.6 Risultati del bilanciamento energetico: confronto tra profili ARERA e profili simulati 68
8.6.1 KPI di comunità 68

8.6.2 KPI per membro e impatto sui prosumer 69
8.6.3 Discussione: interpretazione delle differenze e bias "diurno" del profilo simulato 70
8.7 Valutazione economico-finanziaria: assunzioni di prezzo, incentivi e asset 71
8.7.1 Prezzi di acquisto e vendita 71
8.7.2 Incentivi: TIP e TIAD 71
8.7.3 Asset e ipotesi di CAPEX/OPEX 72
8.7.4 Finanza e fiscalità 72
8.8 Risultati economico-finanziari: confronto tra profili simulati e benchmark ARERA 73
8.8.1 KPI complessivi 73
8.8.2 Conto economico (PnL): principali driver 74
8.8.3 Flussi di cassa e decomposizione del ΔVAN 74
8.8.4 Confronto con simulatori esistenti e chiusura del capitolo . . . 75

9 Conclusioni e sviluppi futuri 77
9.1 Sviluppi futuri del simulatore 77
9.1.1 Produzione: acquisizione dati e generazione automatica di render 3D tramite reti convoluzionali 77
9.1.2 Consumatori: integrazione di nuovi carichi (es. veicoli elettrici) con approccio bottom-up 79
9.1.3 Ottimizzazione: estensione dalla sola batteria all'ottimizzazione degli impianti FV ed eolici 79
9.2 Conclusioni: utilità del simulatore e modularità per riuso e analisi specialistiche 80

Elenco delle figure

2.1 Workflow del simulatore: fasi applicative, punti di integrazione e persistenza su filesystem (sessione, scenari e run). 8

3.1 Profilo orario della temperatura esterna nel 2023 per Doberdò del Lago, generato dal simulatore a partire dai dati Open-Meteo [6]. . . . 20

3.2 Estratto (prime 10 righe) del dataframe orario di sessione con timestamp in UTC, temperatura e variabili di vento. L'obiettivo è mostrare la struttura del dataset utilizzato dal simulatore e le colonne rese disponibili ai moduli a valle. 20

4.1 Schema del workflow di generazione della produzione fotovoltaica nel simulatore. L'orchestrazione avviene in `cer_app` (UI Streamlit), mentre stima geometrica, interrogazione PVGIS e caching sono delegate ai moduli di dominio in `cer_core`. La logica di caching evita chiamate ripetute quando la configurazione non varia. 29

4.2 Interfaccia utente per la definizione delle aree fotovoltaiche. L'utente disegna una superficie installabile e imposta, per ogni area, i parametri geometrici essenziali (tilt e orientamento) che guidano la stima della potenza installabile e la richiesta a PVGIS. 30

4.3 Render 3D del tetto senza pannelli (sinistra) e con pannelli posizionati secondo la logica di packing (destra). La visualizzazione supporta la validazione qualitativa della stima di potenza installabile, evidenziando vincoli geometrici e spaziature. 30

4.4 Esempio di produzione fotovoltaica oraria nel mese di gennaio. Le curve sono riportate per ciascuna area definita dall'utente, evidenziando differenze dovute a geometria, orientamento e potenza installabile stimata. La serie è ottenuta tramite PVGIS e indicizzata in UTC. . . 30

4.5 Esempio di produzione eolica nel mese di gennaio. Le curve sono riportate per ciascuna area/istanza definita in configurazione, mostrando la variabilità tipica dell'eolico guidata dal meteo orario di sessione (UTC) [6]. 30

5.1 Schema concettuale del workflow all'interno della pagina *Consumatori*. 33
5.2 Esempio di profilo carichi base nel mese di gennaio. 39
5.3 Esempio di profilo occupancy-driven nel mese di gennaio. 39
5.4 Esempio di profilo cucina (event-driven) nel mese di gennaio. 39
5.5 Esempio di profilo lavanderia nel mese di gennaio. 39
5.6 Esempio di profilo clima nel mese di gennaio. 39
5.7 Esempio di profilo aggregato nel mese di gennaio. 39

Elenco delle tabelle

4.1 Input principali del modello fotovoltaico nel simulatore. La configurazione è definita in interfaccia e persistita in `producers.json`; la serie oraria è ottenuta tramite PVGIS [7], [8]. 26

4.2 Input principali del modello eolico nel simulatore. La sorgente meteo è il dataset di sessione generato tramite Open-Meteo [6]; la conversione velocità-potenza segue un approccio basato su shear e power curve [10], [11], [13]. 28

5.1 Output tipici del sottosistema consumi nella cache per-consumatore. Tutti i file sono CSV a 15 minuti in kW e condividono lo stesso indice temporale. 33

8.1 Caratteristiche principali delle utenze intervistate (anno base 2023). . 63

8.2 Confronto KPI di comunità (run ARERA vs run simulato). 69

8.3 Confronto KPI per i membri prosumer (run ARERA vs run simulato). 70

8.4 Confronto KPI economico-finanziari complessivi (run ARERA vs run simulato). 73

8.5 Confronto conto economico (Anno 1): run ARERA vs run simulato. . 74

8.6 Somma delle principali voci di conto economico (anni 1–20): run ARERA vs run simulato. 75

8.7 Decomposizione del ΔVAN (attualizzato) tra run simulato e run ARERA. 75

Capitolo 1

Inquadramento e obiettivi del simulatore

1.1 Contesto: comunità energetiche rinnovabili e autoconsumo diffuso

Le Comunità Energetiche Rinnovabili (CER) e, più in generale, le configurazioni di autoconsumo diffuso si collocano nel quadro della transizione energetica e della partecipazione attiva degli utenti al sistema elettrico, abilitando forme di produzione e consumo coordinati in prossimità dei carichi. A livello europeo, tali configurazioni sono inquadrate da due direttive che promuovono la generazione da fonti rinnovabili e la partecipazione dei cittadini e delle comunità energetiche ai mercati dell'energiao (RED II e direttiva sul mercato interno dell'energia elettrica). [1], [2]

Dal punto di vista operativo e regolatorio, un concetto centrale è l'*energia condivisa* su base oraria. Nelle CER italiane la condivisione avviene attraverso la rete pubblica di distribuzione ed è determinata, per ciascuna ora, a partire dai flussi di immissione e prelievo dei punti appartenenti alla configurazione. In particolare, l'energia condivisa nell'ora considerata è definita come il minimo tra (i) l'energia complessivamente immessa in rete dai punti di produzione della configurazione e (ii) l'energia complessivamente prelevata dalle utenze di consumo che ne fanno parte. La definizione è recepita nella disciplina del Testo Integrato Autoconsumo Diffuso (TIAD) e nelle regole operative per l'accesso al servizio, nonché nel decreto attuativo relativo alle configurazioni CACER. [3], [4], [5]

Prima della fase di pianificazione e studio di fattibilità è utile richiamare i principali benefici economici associati alla partecipazione a una CER: (i) il risparmio legato all'autoconsumo, dovuto alla riduzione dell'energia acquistata dalla rete,

(ii) la valorizzazione dell'energia condivisa tramite il meccanismo di incentivazione riconosciuto ai kWh condivisi e (iii) la vendita dell'energia immessa in rete dagli impianti della CER secondo i meccanismi di mercato (ad es. prezzo zonale). In questo quadro, l'analisi di una CER richiede strumenti in grado di quantificare, con granularità temporale adeguata, i flussi energetici dei singoli membri e della comunità e di tradurre tali flussi in risultati economico-finanziari coerenti con assunzioni di scenario (tariffe, incentivi, investimenti). Ad esempio, la decisione di introdurre un sistema di accumulo e il suo dimensionamento dipendono dall'interazione tra profili di produzione, profili di consumo e meccanismi di remunerazione dell'energia condivisa, rendendo utile un simulatore che consenta analisi comparative e di sensitività.

1.2 Obiettivi della tesi e perimetro del lavoro

Obiettivo della tesi. L'obiettivo di questa tesi è analizzare configurazioni di Comunità Energetiche Rinnovabili (CER) e, più in generale, di autoconsumo diffuso, al fine di quantificare i benefici energetici ed economico-finanziari e valutarne la sensibilità rispetto agli input di scenario (profili di consumo e produzione, tariffe e incentivi, investimenti, configurazione dei membri, presenza e taglia di accumulo).

Approccio e strumento. Per perseguire tale obiettivo viene sviluppato un simulatore modulare che, a partire da una configurazione di scenario, ricostruisce i flussi energetici e i corrispondenti flussi economici sia per ciascun membro sia per la comunità nel suo complesso, consentendo una valutazione economico-finanziaria della stessa.

Cosa consente il simulatore. Il simulatore è strutturato come un workflow end-to-end (una *pipeline*)[1] in cui l'output di ciascun modulo costituisce l'input del successivo. Definito lo scenario di riferimento, il tool consente di:

1. generare curve di produzione fotovoltaica (FV) ed eolica su base oraria a partire da PVGIS[2];

[1]Nel seguito, con *pipeline* si indica una sequenza di elaborazioni in cui l'output di un modulo costituisce l'input del successivo, dall'impostazione dello scenario fino ai risultati finali.

[2]PVGIS è l'acronimo di *Photovoltaic Geographical Information System*. Nel contesto di questa tesi viene utilizzato per ottenere stime orarie della producibilità rinnovabile a partire da dati meteo e dalla localizzazione.

2. generare curve di consumo su base a 15 minuti per ciascun membro mediante modellazione *bottom-up*[3] con componenti stocastiche controllate[4];

3. ricostruire i flussi energetici di membro e comunità (prelievi, immissioni, autoconsumo, energia condivisa) e i relativi indicatori;

4. valutare i flussi economici e i principali indicatori finanziari (KPI)[5], includendo una procedura di dimensionamento dell'accumulo in funzione di un criterio economico.

Inoltre, il simulatore è progettato per supportare:

- **riproducibilità**: possibilità di replicare una simulazione a parità di input, assunzioni e parametri stocastici (ad es. seed), rendendo confrontabili scenari e analisi di sensitività;

- **tracciabilità**: registrazione strutturata delle esecuzioni (*run*) e salvataggio persistente degli *artefatti di sessione*[6] (configurazione, assunzioni, output) per consentire verifica e controllo dei risultati;

- **modularità**: capacità di implementare il simulatore come insieme di moduli indipendenti, ciascuno dedicato a un compito ben definito; un componente di orchestrazione coordina l'esecuzione dei moduli e permette di estendere o sostituire singoli blocchi della pipeline senza modificare l'intero processo.

Perimetro del lavoro. Il perimetro è deliberatamente focalizzato su modelli energetici ed economici a complessità controllata, adatti a valutazioni comparative di scenario. Non sono inclusi: un modello elettrico di rete (tensioni, correnti, congestioni), un dimensionamento elettrotecnico di dettaglio (inverter, stringhe, clipping) e un controllo ottimo day-ahead dell'accumulo; tali estensioni sono discusse come sviluppi futuri.

[3]Approccio in cui il profilo aggregato è costruito a partire dal contributo di singole utenze o carichi elementari, invece di partire direttamente da una curva già aggregata.

[4]Si introduce una variabilità casuale per rappresentare l'eterogeneità dei comportamenti; fissando il *seed* (valore iniziale del generatore di numeri casuali) la generazione resta riproducibile.

[5]KPI: *Key Performance Indicators*, indicatori sintetici per confrontare scenari. In questo lavoro si usano, tra gli altri, NPV (VAN: valore attuale netto), IRR (TIR: tasso interno di rendimento) e payback (tempo di ritorno).

[6]Per *artefatti di sessione* si intendono i file e i dataset prodotti durante una specifica esecuzione del simulatore (configurazione dello scenario, parametri/assunzioni e output intermedi/finali). La loro *persistenza* consiste nel conservarli su disco in modo che i risultati possano essere verificati o riprodotti anche a distanza di tempo.

1.3 Assunzioni generali e convenzioni

1.3.1 Orchestrazione della pipeline e responsabilità dei moduli

Il simulatore è organizzato in due livelli distinti:

1. **livello di orchestrazione** (`cer_app`, interfaccia Streamlit), che guida l'utente lungo la pipeline, valida gli input e gestisce la persistenza su disco;
2. **livello di dominio** (`cer_core`), che implementa la logica di calcolo (meteo, produzione, domanda, bilanciamento, economia e ottimizzazione).

Nel seguito, con *orchestratore* si intende una pagina Streamlit che: (i) raccoglie e valida gli input dell'utente, (ii) invoca funzioni e modelli in `cer_core`, (iii) salva su filesystem gli artefatti prodotti, mantenendo coerenza e rintracciabilità dei risultati all'interno di una *sessione*. Una vista d'insieme dell'architettura è riportata in Figura 2.1 (Capitolo 2).

Orchestratore condizioni generali / Home (`cer_app/app.py`). Definisce i *metadati di sessione* (intervallo temporale, localizzazione, seed) e materializza i dataset climatici di riferimento. Invoca `cer_core/condizioni_generali` per geocoding e download/caching meteo e salva in sessione i file meteo (serie orarie e giornaliere) che diventano input canonici per tutte le fasi successive.

Orchestratore produttori (`cer_app/pages/1_Producers.py`). Gestisce la configurazione degli impianti e genera/ingestisce i profili di produzione rinnovabile su griglia oraria. Invoca `cer_core/produttori` (FV ed eolico) e, quando necessario, utilizza le variabili meteo persistite dalla Home. I dettagli modellistici sono discussi nel Capitolo 4.

Orchestratore consumatori (`cer_app/pages/2_Consumatori.py`). Parametrizza e genera i profili di domanda a 15 minuti tramite modellazione bottom-up (carichi base, appliance, occupancy e dipendenze climatiche). Invoca `cer_core/consumatori` e usa, se richiesto dai modelli, le variabili meteo della sessione. Il modello di domanda è descritto nel Capitolo 5.

Orchestratore bilanciamento (`cer_app/pages/3_Bilanciamento_CER.py`). Ricompone produzione e consumo per calcolare i flussi energetici per membro e a livello comunitario (autoconsumo, prelievi, immissioni) e determina l'energia condivisa

su base oraria. Invoca `cer_core/bilanciamento`, che costituisce la *fonte di verità energetica* del simulatore: qui avvengono l'allineamento temporale delle serie, i controlli di coerenza e la produzione degli output energetici persistiti come *run*. Il bilanciamento e la definizione di energia condivisa sono trattati nel Capitolo 6.

Orchestratore valutazione economica (`cer_app/pages/4_Valutazione_Economica.py`). Traduce i risultati energetici in costi/ricavi e KPI finanziari, salvando uno snapshot delle ipotesi (tariffe, incentivi, costi, tasso di sconto) utilizzate nella valutazione. Invoca `cer_core/economics`, che legge gli output del bilanciamento e produce un *run economico* persistito e confrontabile. Quando richiesto, attiva `cer_core/optimization` per esplorare euristiche di dimensionamento/allocazione del BESS in funzione dei KPI (Capitolo 7).

1.3.2 Convenzione temporale e risoluzioni

Il simulatore adotta **UTC come riferimento temporale per tutti gli artefatti persistiti** (input di scenario, cache meteo e output dei run). Questa convenzione riduce ambiguità legate al cambio ora legale (DST) e permette di allineare in modo consistente sorgenti eterogenee (meteo, produzione, consumi). Nel core di calcolo gli indici temporali sono trattati come `DatetimeIndex` *timezone-aware* e normalizzati in UTC.

La pipeline utilizza due griglie temporali principali:

- **15 minuti** per i profili di consumo e per i calcoli di bilancio locale (ad es. logiche intra-ora, inclusa l'eventuale dinamica di accumulo);
- **oraria** per i profili di produzione rinnovabile e per le grandezze comunitarie definite dalla regolazione, in particolare l'energia condivisa.

Quando sono presenti serie su griglie diverse, l'allineamento avviene nel modulo di bilanciamento secondo una regola deterministica. In particolare, la produzione oraria viene portata alla griglia a 15 minuti tramite un'espansione a tratto costante (replica del valore orario sui quattro sotto-intervalli), così da permettere il calcolo dei flussi locali su griglia T15 senza introdurre energia artificiale. Le grandezze legate alla condivisione, invece, sono calcolate e riportate su base *oraria*.

1.3.3 Sessioni, run, riproducibilità e tracciabilità

Ogni simulazione è organizzata in una *sessione*, ovvero una cartella su disco che raccoglie i dati necessari per: (i) ricostruire le scelte di input (scenario), (ii) riutilizzare

i dataset intermedi (es. meteo e serie generate), (iii) confrontare in modo robusto risultati prodotti in esecuzioni successive.

In una sessione si trovano tipicamente:

- **metadati e configurazione**: intervallo temporale, localizzazione e seed (oltre a configurazioni di produttori/consumatori e parametri di scenario);
- **dataset climatici** persistiti (serie orarie e giornaliere) utilizzati come input comune della pipeline;
- **output di sezione**: file prodotti dagli orchestratori (produzione oraria per membro/impianto, consumi a 15 minuti per membro);
- **run energetici** e **run economici**: cartelle/artefatti generati rispettivamente da bilanciamento ed economia, ciascuno corredato da file di output e da metadati di esecuzione.

Riproducibilità. La riproducibilità riguarda la possibilità di rigenerare gli stessi risultati a parità di input. Nel simulatore la componente stocastica è confinata principalmente alla generazione della domanda bottom-up: a ogni sessione è associato un *seed* e, a partire da esso, si ricavano in modo deterministico seed specifici per ciascun consumatore e/o modulo. Di conseguenza, fissati configurazione e parametri, le curve generate e i risultati coincidono.

Tracciabilità. La tracciabilità riguarda la possibilità di confrontare esecuzioni diverse senza perdere lo storico. Per questo motivo i moduli di bilanciamento ed economia registrano ogni esecuzione in un indice aggiornato in modalità *append-only*[7] e mantengono un riferimento al *run attivo* selezionato. Inoltre la valutazione economica salva uno *snapshot* delle assunzioni (tariffe, incentivi, costi, parametri finanziari) usate nella valutazione, in modo che i risultati restino verificabili e confrontabili anche se, in seguito, le tabelle di input vengono aggiornate.

In sintesi, la combinazione di (i) persistenza degli *artefatti di sessione*, (ii) indicizzazione dei run e (iii) controllo della stocasticità consente di ottenere una pipeline **riproducibile** e **tracciabile**, adatta a sperimentazioni iterative su scenari e configurazioni.

[7] Con *append-only* si intende che le nuove esecuzioni vengono aggiunte in coda allo storico senza sovrascrivere quelle precedenti.

Capitolo 2

Architettura del simulatore

Questo capitolo fornisce una descrizione operativa dell'architettura del simulatore e del flusso applicativo end-to-end[1]. L'obiettivo non è discutere nel dettaglio i modelli fisici ed economici, che vengono trattati nei capitoli successivi, bensì definire con precisione come le diverse componenti interagiscono, quali dati vengono prodotti e consumati in ciascuna fase e come tali dati vengono persistiti per garantire riproducibilità e tracciabilità degli esperimenti.

La Figura 2.1 è la rappresentazione canonica del workflow[2]. Essa sintetizza la sequenza delle fasi applicative e mette in evidenza i punti di integrazione tra moduli, inclusa la fase di "ponte" che materializza e trasferisce i profili energetici verso il calcolo di bilanciamento. L'intero capitolo è strutturato come guida di lettura della figura: per ogni blocco vengono chiarite responsabilità, prerequisiti e artefatti su filesystem[3], mentre i rimandi ai capitoli successivi guidano l'approfondimento delle singole metodologie.

2.1 Organizzazione del software

Il simulatore è organizzato secondo una separazione netta tra livello di presentazione e livello di calcolo. La parte applicativa, implementata in Streamlit[4], ha il compito di orchestrare le fasi del workflow, acquisire gli input utente, gestire lo stato della sessione e invocare i moduli di calcolo. La logica di dominio è invece concentrata

[1]Con *end-to-end* si indica l'intero flusso operativo: dall'impostazione degli input fino alla produzione dei risultati finali, includendo tutte le fasi intermedie.

[2]Nel seguito, con *workflow* si intende la sequenza ordinata delle fasi applicative e delle trasformazioni sui dati durante l'esecuzione del simulatore.

[3]Per *filesystem* si intende la struttura di file e cartelle del sistema operativo; qui è usata come supporto per salvare input, output e risultati intermedi.

[4]Streamlit è un framework Python per creare applicazioni web interattive; in questa tesi costituisce l'interfaccia con cui l'utente imposta parametri, avvia le fasi e visualizza i risultati.

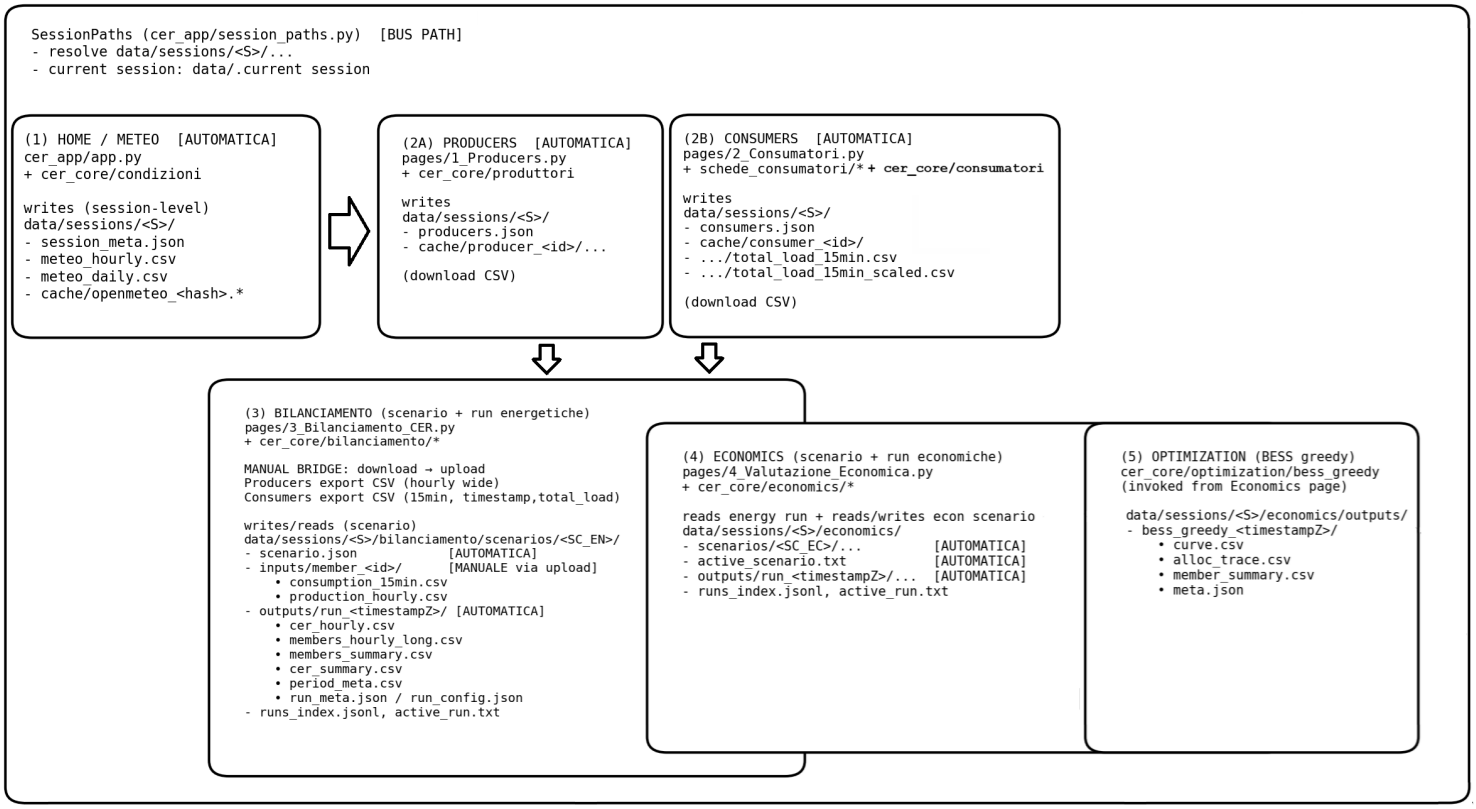

Figura 2.1: Workflow del simulatore: fasi applicative, punti di integrazione e persistenza su filesystem (sessione, scenari e run).

in un core[5] che implementa i modelli e le trasformazioni dei dati, con l'obiettivo di mantenere funzioni quanto più possibile deterministiche e riutilizzabili.

Dal punto di vista del sorgente, questa separazione si concretizza in due macromoduli: `src/cer_app` e `src/cer_core`[6]. Il primo contiene l'applicazione Streamlit, le pagine corrispondenti alle fasi principali e le utilità per la gestione dei percorsi e della sessione attiva. Il secondo contiene i sottosistemi di dominio per l'acquisizione meteo, la generazione della produzione, la modellazione dei consumi, il bilanciamento energetico, la valutazione economica e l'ottimizzazione. In questa architettura il livello applicativo governa l'avanzamento del workflow e materializza gli output in artefatti persistiti, mentre il core è responsabile della correttezza del calcolo e della coerenza dei formati dati.

[5]Nel seguito, con *core* si indica il nucleo del software che contiene la logica di calcolo e le trasformazioni dei dati, indipendente dall'interfaccia utente.

[6]Nomenclatura del progetto: `cer_app` ospita interfaccia e orchestrazione; `cer_core` ospita modelli e funzioni di calcolo.

2.2 Persistenza, sessioni, scenari e run

Una scelta progettuale fondamentale del simulatore è l'adozione del filesystem come meccanismo di persistenza principale. Questa decisione rende trasparente lo stato dell'esecuzione, facilita la riproducibilità dei risultati e permette di ispezionare a posteriori input, output e artefatti intermedi senza dipendere da servizi esterni. La persistenza è strutturata su tre livelli gerarchici, ciascuno con una semantica specifica: *sessione*, *scenario* e *run*.

La *sessione* è l'unità di lavoro di base. Essa contiene i metadati generali dell'analisi e gli artefatti condivisi tra le diverse fasi, come i dataset meteorologici e le configurazioni dei componenti. Operativamente, una sessione corrisponde a una directory dedicata sotto `data/sessions/nome_sessione`. Il simulatore mantiene inoltre un puntatore alla sessione attiva (tramite un file dedicato), in modo da consentire la ripresa del lavoro e la navigazione tra sessioni.

All'interno della sessione vengono persistiti almeno i metadati generali (ad esempio in `session_meta.json`[7]), i dataset meteo (tipicamente in `meteo_hourly.csv`[8] e `meteo_daily.csv`), la configurazione dei produttori (ad esempio `producers.json`) e la configurazione dei consumatori (ad esempio `consumers.json`). Accanto a questi artefatti, una directory di cache[9] consente di memorizzare risultati intermedi e dati derivati (ad esempio output di servizi esterni o profili per singolo componente) allo scopo di ridurre ricalcoli e rendere l'esperienza interattiva.

Lo *scenario* è un contenitore logico usato nelle fasi a valle in cui ha senso ripetere l'elaborazione variando parte degli input mantenendo costante il contesto della sessione. Nel simulatore, il concetto è particolarmente rilevante nel bilanciamento energetico e nella valutazione economica: uno scenario raccoglie input canonici[10] in un formato stabile e indipendente dalla sorgente con cui tali input sono stati generati (ad esempio internamente dal simulatore o forniti dall'utente). La directory di scenario è quindi un "contratto" tra fasi diverse e garantisce che il calcolo possa essere rieseguito in modo controllato.

La *run* è una singola istanza esecutiva di uno scenario. Ogni run produce un insieme coerente di output e viene registrata in un indice append-only, con un puntatore alla run attiva mostrata in interfaccia. Questo meccanismo consente di

[7] JSON (*JavaScript Object Notation*) è un formato testuale strutturato per dati e configurazioni (coppie chiave–valore), usato qui per parametri e metadati.

[8] CSV (*Comma-Separated Values*) è un formato tabellare testuale per lo scambio di dati: ogni riga rappresenta un record e le colonne sono separate da un delimitatore (virgola o punto e virgola).

[9] Per *cache* si intende la memorizzazione di risultati intermedi riutilizzabili, per evitare ricalcoli e rendere l'applicazione più reattiva.

[10] Nel seguito, per *input canonici* si intende una rappresentazione standardizzata (schema e convenzioni fisse) usata come interfaccia stabile tra fasi diverse del simulatore.

confrontare risultati di run successive, preservare la storia delle elaborazioni ed evitare la sovrascrittura di output. La stessa logica viene adottata sia per i risultati del bilanciamento energetico sia per i risultati della valutazione economica.

2.3 Descrizione del workflow

La Figura 2.1 organizza il workflow come una sequenza di fasi che trasformano progressivamente input utente e dataset di contesto in risultati energetici ed economici. Il flusso evidenzia due rami paralleli, dedicati rispettivamente alla generazione della produzione e alla modellazione dei consumi, che convergono in un punto di integrazione prima del bilanciamento. Tale convergenza è implementata tramite un "ponte" che materializza i profili in formato canonico di scenario, in modo da disaccoppiare la generazione dei profili dal calcolo di comunità.

2.3.1 Risoluzione dei percorsi e inizializzazione della sessione

Ogni fase del workflow legge e scrive artefatti in posizioni deterministiche della sessione. Per questo motivo, all'avvio il simulatore risolve la sessione attiva e costruisce una mappa dei percorsi applicativi, che viene poi utilizzata dalle pagine Streamlit e dai moduli core per accedere in modo consistente ai file. Questa componente, evidenziata in figura come *SessionPaths*, è la base che rende riproducibile l'esecuzione: a parità di sessione e di input canonici, l'insieme degli artefatti prodotti è ispezionabile e, se necessario, ricostruibile.

2.3.2 Home e meteo

La fase iniziale del workflow, indicata come *Home/Meteo*, definisce il contesto dell'analisi e produce gli artefatti di base della sessione. In questa fase vengono fissati intervallo temporale e localizzazione geografica, e viene acquisito il dataset meteorologico necessario alle fasi successive. Il risultato è persistito nella sessione come serie temporali meteorologiche, tipicamente a risoluzione oraria, insieme ai metadati che descrivono le scelte effettuate (ad esempio coordinate selezionate e parametri di inizializzazione). I dettagli della procedura di acquisizione e delle convenzioni temporali sono discussi nel Capitolo 3, al quale si rimanda per l'approfondimento.

2.3.3 Produttori

La fase *Producers* costruisce e configura le sorgenti di produzione energetica della comunità. Operativamente, essa acquisisce o deriva i parametri degli impianti

(ad esempio geometrie, orientamenti e caratteristiche nominali) e genera profili di produzione a risoluzione oraria coerenti con il periodo della sessione. La configurazione viene persistita nella sessione, mentre i risultati intermedi e le risposte da servizi esterni vengono salvati in cache per evitare ricalcoli. La scelta di trattare i produttori come ramo indipendente del workflow permette di iterare sulla configurazione degli impianti senza dover rieseguire la modellazione della domanda. L'implementazione e le assunzioni modellistiche sono discusse nel Capitolo 4.

2.3.4 Consumatori

La fase *Consumers* modella la domanda elettrica dei membri. A partire dal contesto della sessione e dai dataset meteo, vengono generati profili di carico tipicamente a risoluzione di 15 minuti, in modo da rappresentare con maggiore dettaglio dinamiche intra-orarie e componenti dipendenti dalla temperatura. Anche in questo caso la configurazione dei consumatori viene persistita nella sessione, mentre profili e componenti intermedi possono essere salvati in cache per singolo membro o per singolo sottosistema. La separazione dal ramo produttori consente di iterare sui parametri comportamentali e sulle assunzioni di domanda mantenendo invariata la produzione. I dettagli dei modelli di consumo e della gestione del seed sono discussi nel Capitolo 5.

2.3.5 Ponte di integrazione e materializzazione degli input

Il punto di convergenza tra i rami produttori e consumatori, indicato in figura come *Manual Bridge*, ha una funzione architetturale precisa: trasformare i profili generati (o alternativamente forniti dall'utente) in input canonici per lo scenario di bilanciamento. Nel simulatore, produttori e consumatori possono produrre output in forme differenti e con convenzioni specifiche; il bilanciamento, al contrario, richiede un set minimo di file tabellari per ciascun membro, con schema e granularità temporale definiti. La materializzazione in formato canonico rende esplicita questa interfaccia e disaccoppia le fasi, permettendo sia l'uso di profili generati internamente sia la sostituzione con profili esterni senza modificare la logica del bilanciamento.

In termini operativi, il ponte consiste nell'esportazione dei profili di consumo (15 minuti) e di produzione (oraria) in file CSV e nel loro caricamento nella struttura di input di uno scenario. Questo passaggio è volutamente evidenziato come manuale nella figura, poiché rappresenta un confine netto tra generazione dei profili e calcolo di comunità, e costituisce il punto in cui l'utente può validare, ispezionare o sostituire gli input.

2.3.6 Bilanciamento energetico

La fase *Bilanciamento* costruisce uno scenario energetico e calcola i flussi della comunità. Lo scenario raccoglie, per ciascun membro, i file di input canonici caricati tramite il ponte: profili di consumo a 15 minuti e, quando presenti, profili di produzione orari. A partire da questi input, il bilanciamento normalizza le convenzioni temporali, aggrega o integra le serie al passo richiesto e produce output energetici tipicamente su base oraria in termini di energia. La fase genera quindi un insieme di output per run, che include risultati per membro e risultati aggregati della comunità, oltre a metadati e snapshot[11] della configurazione della run.

Il bilanciamento adotta un meccanismo di versionamento[12]: ogni esecuzione crea una nuova run e aggiorna un registro append-only, lasciando intatti gli output delle run precedenti; un puntatore identifica la run attiva selezionata in interfaccia. Questa scelta rende possibile l'iterazione controllata sugli input e la comparazione tra configurazioni. Il formalismo e le grandezze energetiche trattate sono approfonditi nel Capitolo 6.

2.3.7 Valutazione economica

La fase *Economics* consuma una specifica run energetica e valuta le prestazioni economiche della comunità e dei singoli membri. In questa fase vengono definiti uno scenario economico, che raccoglie le assunzioni (tariffe, incentivi, costi di investimento e di esercizio) e una run economica che rappresenta l'istanza di calcolo eseguita su una determinata run energetica. Anche qui il simulatore adotta persistenza e versionamento: le assunzioni vengono salvate nello scenario, mentre i risultati (indicatori economici e flussi di cassa) vengono persistiti in una cartella di output dedicata alla run e registrati in un indice. In questo modo è possibile rieseguire il calcolo economico variando le assunzioni senza alterare i risultati energetici e mantenendo la storia delle elaborazioni.

La formulazione degli indicatori e l'impostazione del modello sono discusse nel Capitolo 7.

[11] Copia coerente dei parametri usati in una run, salvata insieme agli output per rendere i risultati verificabili e confrontabili.

[12] Gestione di esecuzioni successive mantenendo lo storico: ogni esecuzione genera una nuova run senza sovrascrivere le precedenti.

2.3.8 Ottimizzazione

L'ultima fase del workflow, indicata come *Optimization*, è un livello applicativo che utilizza la valutazione economica come funzione obiettivo[13]. In questa tesi l'ottimizzazione è finalizzata al dimensionamento di sistemi di accumulo (BESS[14]) attraverso una procedura iterativa che, a parità di scenario economico, valuta varianti di configurazione energetica e confronta gli impatti sugli indicatori economici. Dal punto di vista architetturale, questa fase è una composizione di due passi: un ricalcolo del bilanciamento per la configurazione candidata e una nuova valutazione economica. Gli output dell'ottimizzazione vengono persistiti come artefatti addizionali associati all'analisi economica, mantenendo la stessa logica di tracciabilità dell'esecuzione.

2.4 Convenzioni e contratti dati

Affinché il workflow della Figura 2.1 sia eseguibile in modo robusto, le fasi condividono un insieme di convenzioni sui dati. La convenzione temporale è discussa nel Capitolo 3 (Sezione 3.2) e costituisce un prerequisito dell'interoperabilità[15]: i timestamp sono trattati in UTC e i dataset derivati mantengono coerenza di timezone lungo l'intera pipeline.

Un secondo elemento è la gestione delle risoluzioni temporali. Nel simulatore coesistono serie orarie (meteo e produzione) e serie a 15 minuti (consumi); il bilanciamento produce tipicamente risultati orari in termini di energia, ottenuti integrando la potenza sul passo temporale e aggregando quando necessario. Questo schema consente di modellare la domanda con dettaglio intra-orario, pur mantenendo un bilancio comunitario espresso su base oraria, coerente con le grandezze tipicamente utilizzate per la rendicontazione.

Infine, il ponte di integrazione definisce il contratto minimo dei file di input di scenario. Il fatto che tali file siano tabellari e persistiti in una struttura di directory deterministica rende esplicito il confine tra fasi e permette sia l'audit[16] dei dati sia l'eventuale sostituzione degli input con dati esterni. In altre parole, la sessione non è solo un contenitore di stato applicativo, ma anche un'unità di documentazione

[13] Quantità da minimizzare o massimizzare in un problema di ottimizzazione (criterio con cui si confrontano soluzioni alternative).

[14] *Battery Energy Storage System*: sistema di accumulo elettrico (batterie) che immagazzina energia e la rilascia successivamente.

[15] Capacità di moduli o fasi diverse di scambiarsi dati senza ambiguità grazie a formati e convenzioni condivisi.

[16] Controllo a posteriori di input, output e passaggi di elaborazione per ricostruire come si ottiene un risultato.

sperimentale: essa conserva i dati necessari a ricostruire il percorso che conduce dai parametri iniziali ai risultati energetici ed economici.

2.5 Rimandi ai capitoli successivi

La descrizione architetturale fornita in questo capitolo è intenzionalmente trasversale e orientata al flusso. I dettagli dell'acquisizione meteo e della costruzione del contesto di sessione sono trattati nel Capitolo 3. La modellazione della produzione e i relativi moduli di generazione sono approfonditi nel Capitolo 4, mentre la costruzione dei profili di domanda e la gestione del seed sono discussi nel Capitolo 5. Il bilanciamento energetico, con la formalizzazione dei flussi e delle metriche della comunità, è sviluppato nel Capitolo 6. La valutazione economica e la procedura di ottimizzazione basata sugli indicatori economici sono infine discusse nel Capitolo 7. In questo modo, la Figura 2.1 rimane il riferimento comune che connette le analisi di dettaglio, garantendo coerenza tra implementazione e narrazione.

Capitolo 3

Condizioni generali: configurazione globale e componente climatica

Questo capitolo descrive come il simulatore definisce e materializza le *condizioni generali* di una sessione di simulazione. Questa parte è centrale perché costruisce la **banca dati di base** (dataset e metadati) su cui si appoggiano tutti i moduli a valle: una volta fissati **intervallo temporale**, **localizzazione geografica** e **dataset meteorologico**, ogni analisi successiva (produzioni, consumi, bilanciamenti e valutazioni) eredita in modo coerente gli stessi input, garantendo riproducibilità e tracciabilità.

Nel simulatore, le condizioni generali non sono limitate alla sola temperatura esterna: viene acquisito e persistito un dataset meteo orario multi-variabile che include: (i) temperatura a 2 m dal suolo, (ii) velocità del vento a 100 m, (iii) direzione del vento a 100 m. Queste variabili supportano sia la modellazione della domanda (ad esempio carichi sensibili alle condizioni climatiche) sia i moduli di generazione (ad esempio eolico), senza che i moduli a valle debbano interrogare servizi esterni.

Dal punto di vista implementativo, le condizioni generali derivano dalla cooperazione tra:

- **livello applicativo** (`cer_app/app.py`), responsabile di sessioni, interazione utente e persistenza su disco;
- **livello core** (`cer_core/condizioni_generali/condizioni_climatiche.py`), responsabile delle chiamate a Open-Meteo[1] [6] e della costruzione del dataset meteo.

[1] Open-Meteo è un servizio che fornisce dati meteorologici tramite API; nel simulatore viene utilizzato per ottenere serie orarie multi-variabile a partire da coordinate geografiche e da un intervallo temporale.

3.1 Sessione di simulazione: metadati e persistenza

Ogni scenario è rappresentato da una *sessione*, identificata da un nome e associata a una directory dedicata:

`data/sessions/\{nome_sessione\}/`.

Il simulatore mantiene un puntatore alla sessione corrente nel file `data/.current_session`. Se il puntatore non è presente, l'applicazione inizializza una sessione di default, così da avere sempre un contesto operativo definito.

La logica dei percorsi su disco è standardizzata nel modulo `cer_app/session_paths.py`. In particolare:

- `set_current_session(name)` imposta la sessione corrente e crea la struttura minima su disco;
- `get_current_session_dir()` recupera (o inizializza) la directory della sessione corrente;
- `get_paths(...)` costruisce le convenzioni di path della sessione e crea le cartelle minime necessarie.

È importante notare che questo modulo **non** decide il contenuto dei file: definisce unicamente *convenzioni di percorso* usate in modo consistente dall'applicazione.

All'interno della directory di sessione, il file `session_meta.json` raccoglie i metadati globali:

- `start`, `end`: date di inizio e fine simulazione;
- `seed`: seed numerico per controllare eventuali componenti stocastiche;
- `location`: oggetto che rappresenta la localizzazione geografica.

La localizzazione non è trattata come semplice stringa (nome del comune), ma almeno come coppia (lat, lon). Questa scelta riduce ambiguità (omonimie o errori di digitazione) e rende deterministiche le richieste ai servizi esterni: a parità di coordinate e intervallo temporale, il processo di acquisizione è ripetibile.

3.2 Scelta dell'intervallo temporale e convenzione UTC

Il simulatore utilizza UTC come riferimento temporale end-to-end per tutti i dataset persistiti. Le richieste a Open-Meteo specificano esplicitamente `"UTC"` e i timestamp

sono gestiti come indici con informazione di fuso orario. Questa scelta elimina conversioni implicite legate al tempo locale e semplifica la gestione dei cambi tra ora solare e ora legale, che in timezone locali possono produrre ore duplicate o mancanti.

La granularità di riferimento delle condizioni climatiche è **oraria**. Il dataset meteo è quindi rappresentato su una sequenza di istanti $\mathcal{T}$ a passo costante di 1 h, con copertura completa:

$$\texttt{start 00:00 UTC} \rightarrow \texttt{end 23:00 UTC} \quad \text{(inclusivo)}.$$

Per rendere l'analisi robusta e confrontabile su base stagionale, nella tesi si adotta come scelta consigliata un **intervallo annuale completo**, dal **1 gennaio** al **31 dicembre** (sempre in UTC). In questo modo la banca dati di sessione include l'intera variabilità stagionale e riduce il rischio di bias dovuti a sotto-intervalli non rappresentativi.

3.3 Selezione della località tramite Geocoding API

La selezione della località avviene tramite la Geocoding API[2] di Open-Meteo. In interfaccia l'utente inserisce una stringa di ricerca (ad esempio il nome di un comune); quando la stringa supera una lunghezza minima, la funzione di orchestrazione richiama il core tramite `search_comuni_open_meteo()` e riceve una lista di candidati. Per ciascun candidato sono disponibili nome, area amministrativa e coordinate.

La selezione non è automatica: l'utente conferma esplicitamente il candidato corretto. A seguito della selezione, `app.py` aggiorna `session_meta.json` salvando sia la stringa di ricerca, sia l'oggetto `location` con latitudine e longitudine, che diventa l'unico riferimento utilizzato per l'acquisizione del meteo. Per migliorare la reattività dell'interfaccia, i risultati di ricerca possono essere temporaneamente mantenuti in cache: questa ottimizzazione riduce la latenza percepita, senza alterare la logica della simulazione.

3.4 Acquisizione del meteo orario via Open-Meteo

La generazione del dataset meteorologico di sessione è orchestrata nel livello applicativo dalla funzione `ensure_meteo(meta,force)` in `app.py`. Il comportamento è deterministico rispetto ai metadati di sessione:

[2]Servizio di *geocoding*: a partire da una stringa di ricerca (toponimo) restituisce possibili corrispondenze con coordinate geografiche (latitudine/longitudine) e metadati amministrativi, riducendo ambiguità e omonimie.

1. l'applicazione verifica la presenza nella cartella di sessione dei file `meteo_hourly.csv` e `meteo_daily.csv`;
2. se i file esistono e non è richiesta rigenerazione forzata (`force = False`), vengono riutilizzati;
3. altrimenti, l'applicazione delega al core la costruzione/aggiornamento dei dataset, utilizzando `location`, `start` e `end` presenti in `session_meta.json`.

Questa strategia implementa un caching a livello di sessione: i moduli a valle consumano sempre file persistiti, senza ripetere chiamate API.

Nel core, l'entry-point per la costruzione del meteo orario multi-variabile è `build_meteo_hourly_for_location_open_meteo(...)` in `cer_core/condizioni_generali/condizioni_climatiche.py`. Il builder produce un dataframe[3] con:

- indice temporale UTC, passo 1 h;
- copertura completa da `start 00:00` a `end 23:00` (UTC, inclusivo);
- variabili richieste con convenzioni di colonna uniformi, includendo temperatura (`temp`, in °C), velocità del vento (`wind_speed_100m`, in $\mathrm{m\,s^{-1}}$) e direzione del vento (`wind_direction_100m`, in gradi).

Per coprire in modo affidabile sia la parte storica sia la coda recente/futura, il core combina due endpoint di Open-Meteo [6]:

- **Archive API**[4] per la parte storica;
- **Forecast API** per la porzione più recente e per il futuro ravvicinato.

La separazione avviene rispetto a un cutoff[5] definito come un numero fisso di giorni prima della data corrente (lag di archive): le ore antecedenti al cutoff sono richieste all'Archive API, mentre le ore successive sono richieste alla Forecast API. La Forecast API ha un orizzonte massimo di previsione: se la data di fine simulazione supera tale limite, il core solleva un errore esplicito e richiede di ridurre `end`, evitando di produrre un dataset incompleto o eterogeneo.

[3] Per *dataframe* si intende una struttura dati tabellare (Pandas) composta da righe indicizzate (qui dal timestamp) e colonne corrispondenti alle variabili.

[4] Nel contesto di Open-Meteo, l'*Archive API* fornisce dati storici, mentre la *Forecast API* fornisce dati di previsione e la porzione più recente; i due endpoint differiscono anche per i limiti di copertura temporale.

[5] Soglia temporale che separa la parte richiesta all'Archive API da quella richiesta alla Forecast API; il *lag di archive* è il margine (in giorni) rispetto alla data corrente usato per definire tale soglia.

3.5 Output meteo di sessione e feature engineering

Al termine della pipeline, i dati vengono salvati nella directory della sessione. I file principali sono:

- `meteo_hourly.csv`: serie oraria con timestamp in UTC e variabili meteorologiche; è il riferimento condiviso per i moduli a valle;
- `meteo_daily.csv`: aggregazione giornaliera costruita a partire dalla serie oraria, utile per statistiche sintetiche e per logiche che operano a grana giornaliera.

Nel file giornaliero, per ciascun giorno (definito rispetto a confini giornalieri in UTC) vengono calcolate statistiche descrittive della temperatura (media, minima, massima) e, in modo analogo, statistiche sintetiche per la componente vento (ad esempio medie e valori estremi della velocità). Inoltre viene introdotta una classificazione qualitativa della giornata basata sulla temperatura media: `hot` se $T_{\text{mean}} > 25\,°\text{C}$, `cold` se $T_{\text{mean}} < 5\,°\text{C}$, `mild` altrimenti. Questa etichetta non intende modellare in modo fisico la meteorologia locale, ma rappresenta una forma semplice di feature engineering[6] utile a introdurre contesto climatico discreto in logiche semplificate.

3.6 Organizzazione dei file su disco (audit della sessione)

La sessione è pensata per essere ispezionabile: tutti gli input e gli output sono persistiti sotto `data/sessions/\{nome_sessione\}/`. In particolare, le condizioni generali utilizzano:

- `session_meta.json`: metadati globali (`start`, `end`, `seed`, `location`);
- `meteo_hourly.csv`: dataset meteo orario (UTC, 1 h);
- `meteo_daily.csv`: dataset meteo aggregato giornaliero;
- `cache/`: directory di caching su disco usata dai builder per evitare richieste ripetute a parità di parametri (lat/lon/date/variabili).

Questa organizzazione rende la sessione un'unità autosufficiente: a parità di directory di sessione, è possibile ricostruire dataset e parametri senza dipendere dallo stato dell'interfaccia.

[6] Costruzione di variabili derivate (*feature*) a partire dai dati grezzi, con l'obiettivo di rendere il dataset più utilizzabile in modelli o regole decisionali.

3.7 Visualizzazione e ispezione dei dati

Per documentare la componente climatica e verificare la correttezza della banca dati generata, nel seguito si riportano due output rappresentativi: (i) un profilo orario della temperatura su un intero anno (controllo di coerenza stagionale), (ii) un estratto del dataframe orario, utile a verificare struttura, colonne e unità.

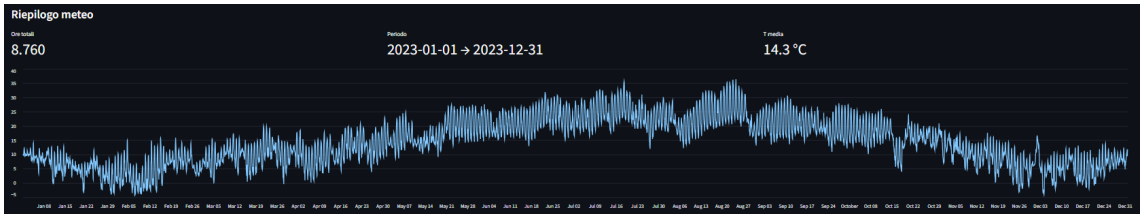

Figura 3.1: Profilo orario della temperatura esterna nel 2023 per Doberdò del Lago, generato dal simulatore a partire dai dati Open-Meteo [6].

	timestamp	temp_mean	temp_min	temp_max	wind_speed_mean	wind_speed_min	wind_speed_max	wind_direction_mean	condition
0	2023-01-01 00:00:00+00:00	10.2708	9.4	11.9	1.7442	0.5	2.72	163.4678	mild
1	2023-01-02 00:00:00+00:00	10.0458	9.4	11.4	1.4638	0.38	3.42	248.3048	mild
2	2023-01-03 00:00:00+00:00	10.0792	7.8	12.2	0.7583	0.1	1.48	174.0717	mild
3	2023-01-04 00:00:00+00:00	9.3708	7.6	14.1	1.9138	0.4	3.44	136.4058	mild
4	2023-01-05 00:00:00+00:00	8.8458	6.9	9.7	2.5329	1.77	3.34	168.6367	mild
5	2023-01-06 00:00:00+00:00	8.2542	3.3	12.1	1.7771	0.1	3.6	135.4958	mild
6	2023-01-07 00:00:00+00:00	8.6	7.4	10.6	1.7275	0.5	3.33	173.3289	mild
7	2023-01-08 00:00:00+00:00	8.4208	5.8	11.1	2.5879	0.14	6.58	99.0061	mild
8	2023-01-09 00:00:00+00:00	9.825	6.4	12.1	4.7071	1.12	9.24	95.8889	mild
9	2023-01-10 00:00:00+00:00	7.0625	3.2	12.7	2.2896	0.5	6.02	267.5201	mild

Figura 3.2: Estratto (prime 10 righe) del dataframe orario di sessione con timestamp in UTC, temperatura e variabili di vento. L'obiettivo è mostrare la struttura del dataset utilizzato dal simulatore e le colonne rese disponibili ai moduli a valle.

Capitolo 4

Produzione energetica

4.1 Scopo del capitolo e inquadramento nella pipeline

La stima della produzione è un passaggio centrale nel simulatore di Comunità Energetiche Rinnovabili (CER), perché determina la disponibilità temporale dell'energia rinnovabile che può essere autoconsumata e/o condivisa, e influenza quindi gli indicatori calcolati nei moduli successivi. Nel progetto sviluppato la produzione è modellata tramite due tecnologie: fotovoltaico ed eolico. La scelta implementativa è orientata a generare profili *orari* plausibili, riproducibili e coerenti con l'intervallo temporale e la localizzazione definiti a livello di *sessione*.

Dal punto di vista architetturale, il simulatore separa chiaramente *orchestrazione UI* e *logiche di dominio*. La componente `cer_app` (Streamlit) gestisce l'interazione utente, la configurazione dei produttori e la persistenza degli artefatti di sessione. La componente `cer_core` incapsula invece il calcolo (stima geometrica, interrogazione PVGIS, conversione eolica da meteo a potenza) e le convenzioni adottate.

Convenzioni di tempo e unità (contratto del modulo). Nel simulatore le serie temporali della produzione sono trattate come **potenza oraria** e indicizzate in **UTC**. In particolare:

- per il fotovoltaico, PVGIS restituisce la potenza `P` in **W** su base oraria [7], [8], [9];
- per l'eolico, la potenza è calcolata internamente in **kW** (a partire dalla power curve in kW) e, quando richiesto, può essere convertita in energia per passo (kWh) tramite integrazione su Δt (in ore).

Quando serve passare da potenza a energia su una griglia oraria, vale la relazione:

$$E_h[\mathrm{kWh}] = \frac{P_h[\mathrm{W}]}{1000} \cdot \Delta t\,[\mathrm{h}], \qquad \Delta t = 1\,\mathrm{h}. \tag{4.1}$$

4.2 Sessione, input comuni e artefatti su disco

Il simulatore è organizzato attorno al concetto di *sessione*, che definisce localizzazione, intervallo temporale e dataset meteorologico associato. Questo capitolo assume che la sessione sia già stata inizializzata nella pagina *Home* e che i file di sessione siano disponibili su filesystem.

4.2.1 File rilevanti per la produzione

Tutta la configurazione e i risultati della produzione sono *session-scoped*, cioè salvati sotto la directory della sessione corrente `data/sessions/<session_name>/`. In particolare:

- `meteo_hourly.csv`: dataset meteo orario indicizzato in UTC, ottenuto tramite Open-Meteo [6]. È la *source of truth* per la produzione eolica.
- `producers.json`: configurazione dei produttori e delle aree (geometrie, orientamento, tilt, parametri FV, parametri eolici). È la *source of truth* della configurazione lato produttori.
- `cache/producer_<id>/...`: cache per produttore, usata per persistire gli output delle elaborazioni (PVGIS ed eolico) ed evitare ricalcoli quando la configurazione non cambia.

4.2.2 Responsabilità dei moduli: UI vs core

Il capitolo fa riferimento ai seguenti moduli (come implementati nello ZIP):

- **UI (Streamlit)**: `cer_app/pages/1_Producers.py` orchestra il flusso di creazione/modifica dei produttori e invoca funzioni di dominio.
- **Core FV**: `cer_core/produttori/produttori.py` implementa persistenza `producers.json`, convenzioni sugli angoli, chiamate a PVGIS e cache PVGIS.

- **Core geometria**: `cer_core/produttori/laf_packing.py` implementa il *packing*[1] dei pannelli nel *Local Area Frame* (LAF)[2]; `cer_core/produttori/roof_3d.py` produce una *mesh* 3D[3] per visualizzazione.

- **Core eolico**: `cer_core/produttori/eolico.py` calcola la produzione oraria a partire da `meteo_hourly.csv` e una power curve (da libreria o override) e gestisce una cache dedicata.

4.3 Workflow della pagina `Producers` e persistenza della configurazione

La pagina `Producers` è l'orchestratore UI della produzione. A livello operativo, il flusso è pensato per rendere esplicito il legame *input utente* → *configurazione persistita* → *serie orarie generate*.

4.3.1 Creazione/modifica dei produttori (CRUD)

La UI consente operazioni di creazione, lettura, modifica ed eliminazione dei produttori (CRUD)[4]. Ogni modifica viene persistita nella sessione tramite le funzioni di utilità del core:

- `load_producers(session_dir)`: carica `producers.json` (lista di produttori), con gestione robusta di file mancanti o corrotti;

- `save_producers(session_dir,producers)`: scrive `producers.json` in modo atomico (tmp + replace).

[1]Nel contesto della geometria, il termine *packing* indica un problema di posizionamento: si collocano elementi (qui pannelli rettangolari) all'interno di un'area rispettando vincoli geometrici (dimensioni, spaziature, margini), al fine di stimare una disposizione ammissibile e il numero installabile.

[2]Con *Local Area Frame* (LAF) si intende un sistema di riferimento locale e planare (in metri) associato all'area di tetto, utile per eseguire operazioni geometriche e metriche (ad es. distanze, aree e posizionamento) evitando le distorsioni delle coordinate geografiche.

[3]Per *mesh* 3D si intende una rappresentazione discreta di una superficie tramite un insieme di poligoni (tipicamente triangoli), usata qui a fini di visualizzazione della geometria del tetto e degli elementi installati.

[4]Acronimo di *Create, Read, Update, Delete*: insieme di operazioni tipiche per la gestione di entità (qui i produttori) in un'applicazione, rispettivamente creazione, lettura, aggiornamento e cancellazione.

4.3.2 Definizione delle aree FV e convenzioni geometriche

Per ciascun produttore l'utente disegna una o più aree di tetto su mappa (poligoni GeoJSON[5]). Le geometrie sono memorizzate in `producers.json` in coordinate geografiche (EPSG:4326[6]) e, quando servono operazioni metriche (ad es. packing), vengono proiettate in una UTM[7] locale determinata automaticamente (funzione `auto_utm_crs`).

Ogni area include inoltre:

- una **classe di orientamento** discreta (`N, NE, E, SE, S, SW, W, NW`);
- un **angolo reale** `angle_deg` in gradi secondo la convenzione interna **0=S, 90=E, 180=N, 270=W**;
- un **tilt** `tilt_deg` in gradi.

Queste scelte non sono solo descrittive: diventano parametri di input del modello FV e sono usate anche per la conversione verso la convenzione richiesta da PVGIS (Sezione 4.4.2).

4.4 Produzione fotovoltaica

La produzione fotovoltaica è ottenuta interrogando PVGIS (endpoint *seriescalc*), che restituisce una serie oraria di potenza in funzione di localizzazione e parametri impiantistici [7], [8], [9]. Nel simulatore l'orchestrazione (UI, scelta aree, parametri) è separata dalla logica di calcolo (stima kWp e chiamata PVGIS).

4.4.1 Stima della potenza installabile: packing nel Local Area Frame

Una volta definite le aree, il simulatore stima il numero di moduli installabili mediante un algoritmo di *packing* che lavora in un sistema metrico locale (LAF), così da operare in metri e non in gradi geografici. La funzione invocata dalla UI è `pack_panels_laf_lonlat`, che:

- proietta l'area in una UTM locale;

[5]Formato standard basato su JSON per rappresentare geometrie geografiche (punti, linee, poligoni) e relativi attributi; nel simulatore è usato per descrivere le aree di tetto in modo interoperabile.

[6]Codice EPSG del sistema di riferimento WGS84: latitudine e longitudine espresse in gradi.

[7]*Universal Transverse Mercator*: famiglia di proiezioni cartografiche che esprimono le coordinate in metri (per zona), utile per calcoli metrici su distanze e aree.

- posiziona pannelli rettangolari rispettando dimensioni del pannello e parametri di spaziatura (gap, margin);
- restituisce i footprint dei pannelli (poligoni) e conteggi (`n_panels`).

Per garantire riproducibilità e facilità di rendering, i footprint dei pannelli possono essere salvati direttamente in `producers.json` (campo `panels_geojson`).

4.4.2 Interrogazione PVGIS e convenzione degli angoli

Per ogni area selezionata viene richiesto a PVGIS un profilo orario di potenza. Nel core questa operazione è incapsulata in `pvgis_series_hourly`, che restituisce un `DataFrame` indicizzato in UTC con colonna `P` (W).

Un punto critico per la coerenza del modello riguarda la conversione degli angoli. Il simulatore usa l'azimut con convenzione:

$$0^\circ = \text{Sud},\ 90^\circ = \text{Est},\ 180^\circ = \text{Nord},\ 270^\circ = \text{Ovest}.$$

PVGIS utilizza invece il parametro `aspect` con 0=S, valori positivi verso Ovest e negativi verso Est. La conversione è gestita esplicitamente dalla funzione `pvgis_aspect_from_project_azimuth` prima della chiamata all'API, evitando ambiguità nella definizione dell'orientamento.

4.4.3 Caching della produzione FV

Per evitare chiamate ripetute a PVGIS quando la configurazione non varia, il simulatore implementa una cache per produttore sotto `<session>/cache/producer_<id>/pvgis/`. La UI utilizza:

- `producer_cache_dir(session_dir,producer_id)` per risolvere e creare la directory di cache del produttore;
- `cache_pvgis_load` / `cache_pvgis_save` per leggere/scrivere i risultati della richiesta PVGIS.

In questo modo, a parità di input (geometria, tilt, azimut, kWp, perdite, tecnologia e anno PVGIS), il profilo orario viene riutilizzato senza contattare nuovamente il servizio esterno.

4.4.4 Input del modello fotovoltaico

Per rendere trasparenti le assunzioni e favorire la riproducibilità, la Tabella 4.1 sintetizza i principali input utilizzati nella generazione della produzione fotovoltaica.

Tabella 4.1: Input principali del modello fotovoltaico nel simulatore. La configurazione è definita in interfaccia e persistita in `producers.json`; la serie oraria è ottenuta tramite PVGIS [7], [8].

Categoria	**Input**
Geometria area	Poligono area (GeoJSON, EPSG:4326); proiezione metrica (UTM) per packing e calcolo aree; punto rappresentativo per interrogazione PVGIS.
Orientamento	Inclinazione (tilt) e azimut (`angle_deg`) secondo convenzione interna 0=S, 90=E, 180=N, 270=W; conversione in `aspect` per PVGIS.
Packing pannelli	Dimensioni pannello (`panel_w`, `panel_h`); `gap` e `margin`; criteri di copertura (`coverage_min_pct`); footprint pannelli opzionale in `panels_geojson`.
Potenza di picco	Numero pannelli stimato (`n_panels`); potenza nominale per pannello (`wpp`); potenza totale in kWp per area e aggregabile a produttore.
Assunzioni impiantistiche	Perdite globali (`system_loss`, %); tecnologia FV (`tech`); anno/i di riferimento selezionati in interfaccia.
Output	Serie oraria di potenza `P` (W) indicizzata in UTC; caching in `cache/producer_<id>/pvgis/`.

4.5 Produzione eolica

A differenza del fotovoltaico, la produzione eolica è calcolata *internamente* a partire dal meteo di sessione. Il modulo `cer_core/produttori/eolico.py` dichiara esplicitamente che **non scarica meteo esterno**: la *source of truth* è il file `meteo_hourly.csv`, generato in sessione tramite Open-Meteo [6].

4.5.1 Pipeline tecnica: meteo → velocità a quota mozzo → power curve

Il calcolo segue una pipeline di tipo performance-based (in linea con approcci ingegneristici basati su curva di potenza) [10], [11]:

1. lettura del meteo orario e selezione della colonna di velocità del vento (ad es. `wind_speed_100m`);

2. estrapolazione della velocità a quota mozzo tramite power-law (shear):

$$V_{\text{hub}} = V_{\text{ref}} \left(\frac{H_{\text{hub}}}{H_{\text{ref}}} \right)^{\alpha}, \tag{4.2}$$

dove H_{ref} è la quota di riferimento del meteo, H_{hub} la quota mozzo e α l'esponente configurabile [10], [12];

3. conversione velocità→potenza tramite power curve discreta e interpolazione lineare;

4. applicazione di perdite globali (%) per ottenere potenza netta;

5. aggregazione di N turbine abilitate.

La definizione della power curve è coerente con l'idea di prestazione misurata e tabulata (pur senza implementare correzioni avanzate, ad es. densità dell'aria) e si allinea al razionale degli standard sulle prestazioni [13].

4.5.2 Libreria turbine e override della curva

Il simulatore include una libreria di modelli di turbina in `wind_turbine_library.json`. Il core la carica con `load_wind_turbine_library`. Per ogni turbina configurata, la power curve viene risolta con priorità:

1. `power_curve_override` (se presente);

2. `power_curve` (legacy);

3. `model_id` che punta a un modello nella libreria.

4.5.3 Caching della produzione eolica

Anche per l'eolico è prevista una cache per produttore, sotto `<session>/cache/producer_<id>/eolico/`. La chiave di cache è costruita in modo da invalidare correttamente i risultati quando cambiano gli input rilevanti. In particolare, il modulo considera:

- l'hash del contenuto di `meteo_hourly.csv`;

- un hash canonico dei parametri delle turbine (JSON);

- l'hash della libreria turbine;

- una versione dell'algoritmo.

La UI utilizza funzioni dedicate (ad es. `build_eolico_cache_key`, `cache_eolico_load`, `get_or_compute_eolico_hourly`) per caricare o (se necessario) ricalcolare la serie oraria.

Tabella 4.2: Input principali del modello eolico nel simulatore. La sorgente meteo è il dataset di sessione generato tramite Open-Meteo [6]; la conversione velocità-potenza segue un approccio basato su shear e power curve [10], [11], [13].

Categoria	**Input**
Meteo di sessione	`meteo_hourly.csv` (UTC); colonna velocità vento (es. `wind_speed_100m`); indicizzazione temporale in UTC [6].
Quote e shear	Quota di riferimento H_{ref}; quota mozzo H_{hub}; esponente α del power-law.
Turbina	Modello turbina (`model_id`); curva di potenza discreta (v, p) da libreria o override; numero di turbine abilitate.
Perdite	Perdite globali percentuali (%); criteri di aggregazione (per turbina, per area/istanza, per produttore).
Output	Serie oraria di potenza (kW) o energia per passo (kWh) a seconda della modalità selezionata; caching in `cache/producer_<id>/eolico/`.

4.5.4 Input del modello eolico

La Tabella 4.2 riassume gli input del modello eolico, distinguendo tra dati meteorologici di sessione e parametri specifici della turbina.

4.6 Output prodotti e utilizzo nei moduli successivi

L'output della fase di produzione consiste in una o più serie orarie (una per area/istanza, aggregabili a livello di produttore) salvate in cache e richiamabili nei passaggi successivi. A livello di pipeline:

- **la configurazione** (geometrie, parametri FV/eolico) è sempre recuperabile da `producers.json`;
- **le serie orarie** sono recuperate dalla cache (PVGIS/eolico) se già calcolate;
- i moduli successivi (bilanciamento CER, valutazione economica) consumano queste serie come input, mantenendo l'allineamento temporale in UTC.

4.7 Verifiche qualitative e limiti del modello

La produzione modellata è progettata per essere verificabile e riproducibile a partire dagli artefatti di sessione. A livello qualitativo:

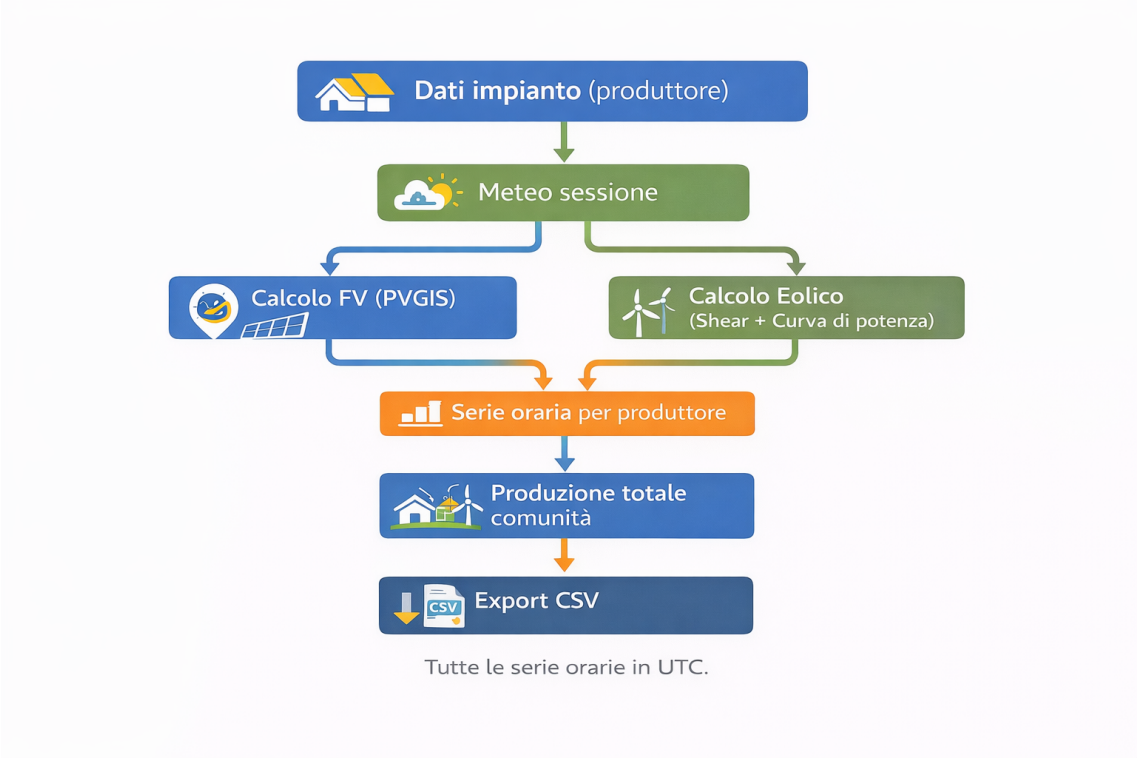

Figura 4.1: Schema del workflow di generazione della produzione fotovoltaica nel simulatore. L'orchestrazione avviene in `cer_app` (UI Streamlit), mentre stima geometrica, interrogazione PVGIS e caching sono delegate ai moduli di dominio in `cer_core`. La logica di caching evita chiamate ripetute quando la configurazione non varia.

- nel fotovoltaico ci si attende produzione nulla nelle ore notturne e massimi diurni; differenze tra aree derivano da orientamento, tilt e potenza installabile stimata (packing);
- nell'eolico la variabilità del profilo segue direttamente la dinamica del vento nel meteo orario di sessione [6]; i parametri H_{hub}, α e le perdite consentono analisi di sensibilità controllate.

Il capitolo descrive un compromesso metodologico coerente con l'obiettivo del simulatore: privilegiare trasparenza delle assunzioni, riproducibilità e coerenza temporale rispetto a modelli fisici di dettaglio che richiederebbero input addizionali (ad esempio profili verticali completi, densità dell'aria e modellazione dei wake effects).

4.8 Figure del capitolo

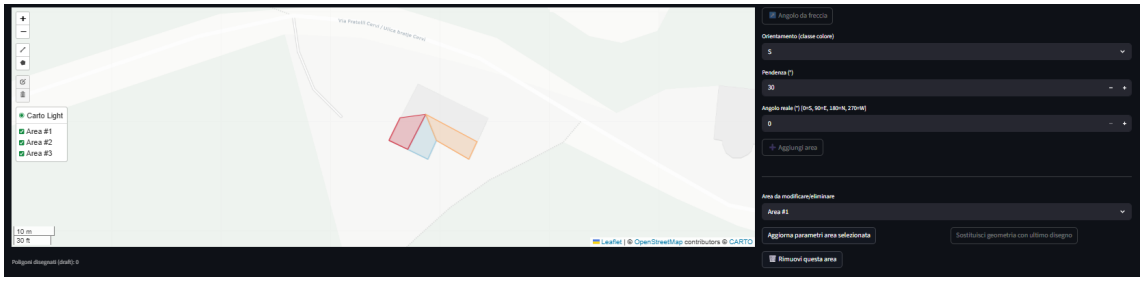

Figura 4.2: Interfaccia utente per la definizione delle aree fotovoltaiche. L'utente disegna una superficie installabile e imposta, per ogni area, i parametri geometrici essenziali (tilt e orientamento) che guidano la stima della potenza installabile e la richiesta a PVGIS.

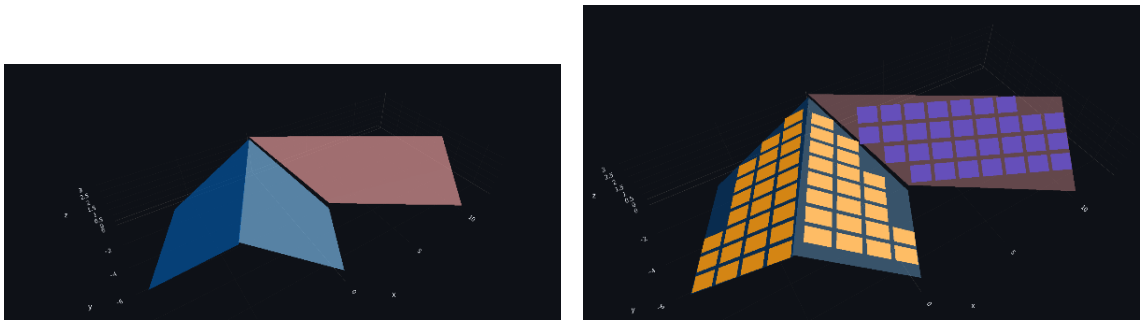

Figura 4.3: Render 3D del tetto senza pannelli (sinistra) e con pannelli posizionati secondo la logica di packing (destra). La visualizzazione supporta la validazione qualitativa della stima di potenza installabile, evidenziando vincoli geometrici e spaziature.

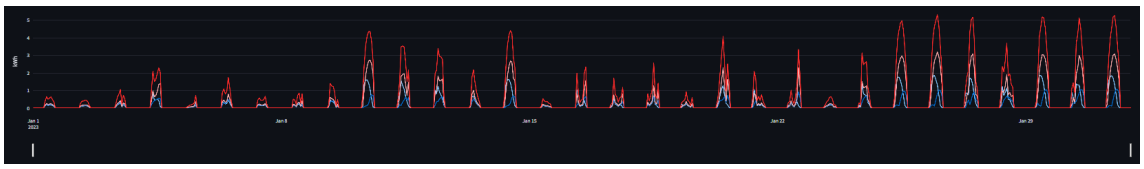

Figura 4.4: Esempio di produzione fotovoltaica oraria nel mese di gennaio. Le curve sono riportate per ciascuna area definita dall'utente, evidenziando differenze dovute a geometria, orientamento e potenza installabile stimata. La serie è ottenuta tramite PVGIS e indicizzata in UTC.

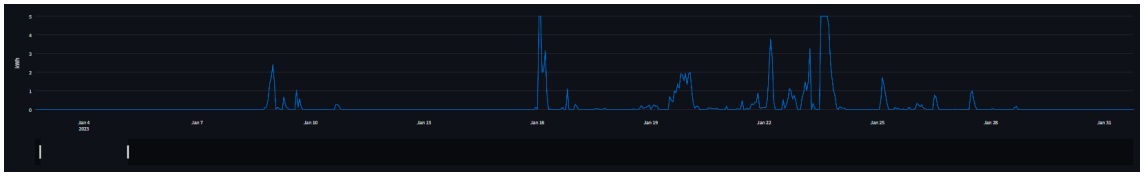

Figura 4.5: Esempio di produzione eolica nel mese di gennaio. Le curve sono riportate per ciascuna area/istanza definita in configurazione, mostrando la variabilità tipica dell'eolico guidata dal meteo orario di sessione (UTC) [6].

Capitolo 5

Domanda elettrica: modellazione bottom-up dei consumi

5.1 Modello e integrazione nella pipeline

5.1.1 Inquadramento e requisiti del sottosistema domanda

Il sottosistema *domanda* genera, per ciascun consumatore, un profilo di potenza elettrica $P(t)$ su base temporale a 15 minuti, coerente con le caratteristiche dell'utenza (numero di residenti, dotazione di apparecchi, abitudini), con le condizioni esterne (meteo) e con vincoli di riproducibilità e tracciabilità degli output. L'impostazione adottata è *bottom-up*: il profilo totale non viene stimato direttamente, ma emerge dalla somma di componenti modellate separatamente e interpretabili (carichi base, occupancy, cucina, lavanderia, clima), seguendo approcci consolidati in letteratura [14], [15], [16]. In ambito nazionale, una modellazione parametrica e modulare è coerente con analisi e dati di riferimento sui consumi residenziali [17], [18] (si veda anche [17] e [18]). Inoltre, l'integrazione tra modelli di domanda e contesto energetico complessivo è in linea con approcci integrati descritti in letteratura [19].

L'obiettivo progettuale non è massimizzare la complessità fisica di ciascun sottosistema, bensì costruire un generatore di profili adatto a scenari *what-if*[1]. Il principio guida è che una variazione di input (ad esempio dotazione, vincoli orari, numero di cicli settimanali, setpoint[2] e impianto) deve tradursi in una variazione osservabile e spiegabile della curva. Per questo il simulatore adotta una scomposizione in moduli che producono output intermedi persistiti, così da consentire verifica, debug e riuso lungo la pipeline.

[1]Per *what-if* si intende un'analisi di scenario in cui l'utente varia in modo controllato alcuni parametri d'ingresso per osservare l'effetto sul profilo risultante.

[2]Valore obiettivo (ad es. temperatura interna) che un regolatore tenta di mantenere.

5.1.2 Razionale modellistico: perché questa implementazione

Un approccio top-down, basato su curve medie di categoria o su regressioni aggregate, può essere efficace quando si dispone di calibrazioni robuste su dataset esterni e l'obiettivo è riprodurre statistiche medie. Nel contesto del simulatore CER, invece, il requisito dominante è la tracciabilità del legame tra scelte di scenario e forma temporale della domanda. La scomposizione bottom-up consente di distinguere la componente quasi-stazionaria (carichi base) dai contributi comportamentali (occupancy, cucina, lavanderia) e da quelli termo-dipendenti (clima), evitando che il profilo sia un "oggetto opaco". Questo compromesso è coerente con la letteratura sui generatori di profilo residenziale ad alta risoluzione basati su occupazione, attività e stocasticità [15], [20], [21].

La scelta della granularità a 15 minuti è funzionale al bilanciamento energetico e all'allineamento con le grandezze della simulazione CER; la presenza di input meteo orario viene gestita mediante riallineamento su timebase[3] a 15 minuti, così che tutti i moduli operino su un indice temporale comune e univoco. In questa cornice, la modularità non è soltanto una scelta di organizzazione del codice, ma un vincolo metodologico: ogni modulo deve avere un set di input ridotto e controllabile in UI e deve produrre un output verificabile, preservando riproducibilità e separabilità dei contributi.

5.1.3 Struttura del workflow e artefatti prodotti

La Fig. 5.1 schematizza il workflow della pagina *Consumatori*. L'utente definisce la configurazione del consumatore, il sistema genera le curve di componente su base a 15 minuti, salva gli artefatti nella cache dedicata e infine produce il profilo totale, eventualmente riscalato su un target annuo. La persistenza degli input e la cache degli output costituiscono un punto chiave: rendono esplicito il collegamento tra input, algoritmi e curve risultanti e consentono di ispezionare il contributo di ciascun modulo in modo indipendente.

5.1.4 File generati e convenzioni di output

La Tab. 5.1 riassume gli output tipici prodotti nella cache del consumatore (risoluzione 15 minuti). L'aggregato totale e la versione riscalata (se attivata) sono salvati rispettivamente in `total_load_15min.csv` e `total_load_15min_scaled.csv`. Tutti i file sono serie temporali in kW con lo stesso indice temporale, condizione necessaria

[3] Risoluzione/granularità temporale dell'indice su cui sono discretizzate e allineate le serie.

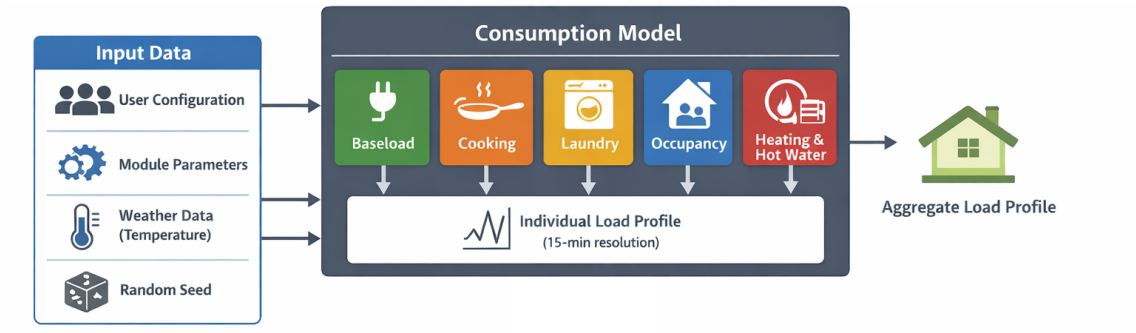

Figura 5.1: Schema concettuale del workflow all'interno della pagina *Consumatori*.

per rendere la somma delle componenti un'operazione ben definita. Nel seguito, l'acronimo ACS[4] indica l'acqua calda sanitaria, inclusa nel modulo clima.

Tabella 5.1: Output tipici del sottosistema consumi nella cache per-consumatore. Tutti i file sono CSV a 15 minuti in kW e condividono lo stesso indice temporale.

Componente	**File**	**Contenuto**
Carichi base	`baseload.csv`	Always-on, standby e cicli frigo/freezer
Occupancy	`occupancy_15min.csv`	Presenza/attività e carichi guidati da occupancy (es. luci/elettronica)
Cucina	`kitchen.csv`	Eventi legati ai pasti e uso elettrodomestici di cucina
Lavanderia	`laundry_total.csv`	Cicli lavatrice/asciugatrice allocati su finestre ammissibili
Clima	`climate.csv`	Carico termo-dipendente (riscaldamento/raffrescamento/ACS)
Totale	`total_load_15min.csv`	Somma delle componenti riallineate su indice comune
Totale riscalato	`total_load_15min_scaled.csv`	Profilo totale moltiplicato per fattore di scala energetico

5.2 Moduli di generazione della domanda

5.2.1 Carichi base: always-on, standby e cicli (Baseload)

I carichi base (*baseload*)[5] rappresentano la componente a bassa variabilità della domanda. In questa categoria rientrano i consumi *always-on*[6], i consumi di *standby*[7]

[4]ACS: *Acqua Calda Sanitaria*.

[5]Componente di domanda relativamente stabile e poco variabile nel breve periodo, non direttamente legata a eventi/attività specifiche.

[6]Carichi permanentemente alimentati e pressoché continui (ad es. router o elettronica in funzione continua).

[7]Potenza assorbita da dispositivi inattivi ma pronti all'uso.

e i dispositivi che seguono un comportamento ciclico di tipo ON/OFF[8], come frigo e freezer. La forma della curva risultante è tipicamente caratterizzata da un livello quasi costante, con oscillazioni dovute ai cicli e con assenza di picchi fortemente legati all'attività umana, che vengono invece modellati dai moduli comportamentali. Tale impostazione è coerente con la letteratura sui profili bottom-up e consente di separare in modo netto la componente "strutturale" della domanda dai carichi evento-guidati [14], [15].

Listing 5.1: Pseudo-codice compatto per la generazione dei carichi base (baseload).

```
P_const_W = sum(always_on_W) + sum(standby_W)
P_cycle_W = 0
for device in [fridge, freezer_if_present]:
    state = init_state(seed)
    for t in INDEX:
        state = markov_ON_OFF_step(state, seed, t)
        P_cycle_W[t] += (state==ON) * P_device_W
P_baseload_kW = (P_const_W + P_cycle_W) / 1000
save_csv("baseload.csv", P_baseload_kW)
```

Nel Listato 5.1 la funzione `markov_ON_OFF_step` implementa un passo di catena di Markov[9] per descrivere la commutazione ON/OFF dei carichi ciclici.

5.2.2 Occupancy-driven: presenza, attività e illuminazione

Il modulo di *occupancy*[10] costruisce una rappresentazione temporale della presenza e dell'attività domestica, a partire da una settimana tipo e da parametri di variabilità, con l'obiettivo di evitare profili eccessivamente deterministici. In letteratura l'occupazione è riconosciuta come uno dei principali driver dei consumi residenziali ad alta risoluzione e costituisce la base per la generazione di carichi condivisi e per la modellazione dell'illuminazione [15], [21]. La componente luce è modellata combinando la condizione di presenza con un proxy[11] di "buio", coerentemente con approcci classici che legano accensione delle luci a presenza e condizioni di illuminazione ambientale [22], [23].

Listing 5.2: Pseudo-codice compatto per la generazione occupancy-driven.

[8] Dinamica a due stati (acceso/spento) con commutazioni nel tempo.

[9] Processo stocastico in cui la probabilità del prossimo stato dipende soltanto dallo stato corrente.

[10] Rappresentazione (tipicamente discreta) della presenza/assenza e, più in generale, dell'attività domestica; *occupancy-driven* indica carichi la cui attivazione o potenza dipendono dallo stato di presenza/attività.

[11] Variabile surrogata utilizzata come indicatore indiretto di una grandezza non misurata direttamente.

```
occ_local = weekly_matrix_to_series(matrix, timezone)
occ_local = add_jitter(occ_local, variability, seed)
occ = local_to_UTC(occ_local).reindex(INDEX).ffill()

for t in INDEX:
    P_small_W[t] = P_min if occ[t]==AWAY else P_min +
       sample_sessions(seed, t)

P_light_W = present(occ) * darkness_proxy(t) * P_lighting_W
save_csv("occupancy_15min.csv", (P_small_W + P_light_W)/1000)
```

Nel Listato 5.2 il termine `add_jitter` introduce un jitter[12] controllato; l'operatore `ffill`[13] mantiene coerenza dell'indice temporale dopo il riallineamento.

5.2.3 Cucina: modello event-driven basato su pasti

I consumi di cucina sono descritti come somma di eventi, in quanto l'uso degli elettrodomestici è naturalmente ancorato ad attività e routine (preparazione dei pasti, pulizia). L'approccio *event-driven*[14] consente di generare picchi concentrati in finestre temporali compatibili con pranzo e cena e di riprodurre cicli più lunghi e regolari per elettrodomestici come lavastoviglie. Questo impianto è coerente con l'uso di modelli basati su attività/*time-use*[15] nella generazione di profili residenziali [15], [20]. Per gli aspetti di efficienza e classificazione energetica si fa riferimento anche alle normative tecniche applicabili, che motivano l'attenzione alle classi e ai consumi tipici di ciclo [24], [25].

Listing 5.3: Pseudo-codice compatto per la generazione dei consumi di cucina (event-driven).

```
events = []
for day in period:
    meals = sample_meals(meal_matrix, seed, day)
    for meal in meals:
        events += maybe_event("oven", meal.time, params, seed)
```

[12]Piccola perturbazione casuale (di tempi e/o valori) usata per rappresentare variabilità e ridurre determinismo.

[13]*Forward fill*: riempimento dei valori mancanti propagando in avanti l'ultimo valore valido.

[14]Impostazione in cui i consumi sono generati da eventi discreti (accensioni/cicli) sovrapposti nel tempo, anziché da una dinamica continua.

[15]Approcci che utilizzano dati o diari sull'uso del tempo (*time-use surveys*) per inferire routine, attività e probabilità di utilizzo degli apparecchi.

```
        events += maybe_event("induction", meal.time, params, seed)
        events += maybe_event("microwave", meal.time, params, seed)
        events += correlated_event("hood", tied_to="cooking", seed=seed)
    events += maybe_cycle("dishwasher", start_after=meals.last, params=params, seed=seed)

P_kitchen_W = superpose(events, INDEX)
save_csv("kitchen.csv", P_kitchen_W/1000)
```

5.2.4 Lavanderia: cicli discreti con vincoli di scheduling

Lavatrice e asciugatrice sono modellate come cicli discreti che avvengono un numero di volte per settimana dipendente dalla configurazione dell'utenza. Un elemento centrale è la presenza di vincoli di *scheduling*[16], espressi mediante finestre ammissibili di avvio: questo riduce la casualità "pura" e rende il profilo più coerente con abitudini realistiche e con strategie di spostamento del carico. La curva risultante appare dunque episodica, con blocchi di potenza associati ai cicli e con una distribuzione temporale dominata dai vincoli. Anche in questo caso, le considerazioni su efficienza e consumi di ciclo possono essere motivate richiamando le normative e le classi energetiche [26], [27].

Listing 5.4: Pseudo-codice compatto per la generazione lavanderia con scheduling vincolato.

```
n_cycles = cycles_from_week_rate(cycles_per_week, period)
slots = start_matrix_to_allowed_slots(start_matrix)
starts = sample_starts(slots, n_cycles, seed)

P_W = 0
for s in starts:
    P_W += place_cycle(washer_profile, INDEX, start=s)
if dryer_present:
    starts_d = sample_starts(slots_dryer, n_dryer_cycles, seed)
    for s in starts_d:
```

[16] Allocazione temporale dei cicli entro finestre ammissibili di avvio, eventualmente con criteri di scelta/ottimizzazione o vincoli di abitudine.

```
        P_W += place_cycle(dryer_profile, INDEX, start=s)

save_csv("laundry_total.csv", P_W/1000)
```

5.2.5 Clima: modello termo-elettrico compatto e dipendenza dal meteo

Il modulo clima introduce la dipendenza della domanda dalle condizioni esterne e dalle caratteristiche dell'edificio/impianto. L'idea è tradurre temperatura esterna e setpoint in una richiesta termica, quindi convertire tale richiesta in potenza elettrica in funzione delle prestazioni del generatore. L'impiego di modelli compatti a parametri concentrati è una scelta comune quando l'obiettivo è catturare inerzia e dipendenza dal meteo senza ricorrere a simulazioni edilizie dettagliate; la letteratura su modelli *lumped* e RC[17] costituisce il riferimento metodologico [28]. La conversione termico-elettrica tramite COP[18], eventualmente dipendente dalla temperatura esterna, è coerente con formulazioni semplificate utilizzate per pompe di calore in studi applicativi [29].

Listing 5.5: Pseudo-codice compatto per la generazione del carico clima (termo-elettrico).

```
T_out = interpolate_to_15min(meteo_hourly.temperature, INDEX)
T_in = T_init
for t in INDEX:
    Q_loss = (T_in - T_out[t]) / R_env
    Q_ctrl = control(setpoint, schedule, T_in, deadband)
    Q_hvac = Q_loss + Q_ctrl
    COP = performance_map(T_out[t], device_type)
    P_elec_kW[t] = max(0, Q_hvac / COP) / 1000
    T_in = update_RC_state(T_in, Q_hvac, C_th, dt)
save_csv("climate.csv", P_elec_kW)
```

Nel Listato 5.5 il controllo utilizza una *deadband*[19]; la funzione `performance_map`[20] rappresenta la dipendenza delle prestazioni dalle condizioni esterne.

[17] *Lumped*: modelli a parametri concentrati, in cui proprietà distribuite (inerzia, dispersioni) sono aggregate in pochi parametri equivalenti. Modello RC: rappresentazione termo-elettrica a rete resistenza–capacità (R–C) per descrivere dispersioni e accumulo termico.

[18] COP (*Coefficient of Performance*): rapporto tra potenza termica resa e potenza elettrica assorbita dal generatore, spesso funzione delle condizioni operative.

[19] Banda di insensibilità attorno al setpoint entro cui il regolatore non applica correzioni, per evitare commutazioni frequenti.

[20] Mappa prestazionale (tabellata o parametrica) che associa condizioni operative (ad es. temperatura esterna) alle prestazioni del generatore, ad esempio in termini di COP.

5.3 Aggregazione, riscalatura e interpretazione dei risultati

5.3.1 Aggregazione del profilo totale

Il profilo totale del consumatore è ottenuto sommando le componenti calcolate dai moduli, tutte espresse sulla medesima base temporale. L'aggregazione non è soltanto un'operazione matematica, ma un passaggio metodologico che preserva interpretabilità: ciascuna componente è un artefatto persistito in cache, e il profilo totale è il risultato della loro composizione. In questo modo, eventuali anomalie possono essere ricondotte al modulo responsabile e corrette agendo sugli input del solo sottosistema coinvolto.

5.3.2 Riscalatura su target energetico annuo

In molti casi è utile imporre un vincolo energetico complessivo, ad esempio un consumo annuo target E_{ann} (kWh), mantenendo però la forma temporale del profilo generato. In questa impostazione la riscalatura viene implementata come moltiplicazione per uno scalare unico, calcolato sul periodo simulato, così da preservare pattern e stagionalità. Indicando con Δt la durata del passo temporale in ore e con $P(t)$ la potenza in kW, l'energia grezza sul periodo è $E_{\text{raw}} = \sum_t P(t)\Delta t$. Se il periodo simulato copre N_d giorni, il target sul periodo è posto pari a $E_{\text{target}} = E_{\text{ann}} \cdot N_d/365$; il fattore di scala è quindi $\alpha = E_{\text{target}}/E_{\text{raw}}$ e la curva riscalata è $P_{\text{scaled}}(t) = \alpha P(t)$. Questo meccanismo è intenzionalmente semplice: mantiene inalterata la struttura temporale prodotta dai moduli e consente di calibrare il livello energetico senza introdurre ulteriori gradi di libertà.

5.4 Esempi di profili e discussione qualitativa

La Fig. 5.2 mostra un esempio di carichi base nel mese di gennaio, evidenziando una componente quasi costante con variazioni cicliche. La Fig. 5.3 riporta un profilo occupancy-driven con modulazione legata alla presenza e all'illuminazione. Le Fig. 5.4 e 5.5 mostrano contributi event-driven (cucina e lavanderia) con picchi localizzati in finestre temporali coerenti con abitudini e vincoli di scheduling. La Fig. 5.6 evidenzia l'effetto della dipendenza dal meteo sul carico clima. Infine, la Fig. 5.7 mostra il profilo aggregato, ottenuto come composizione delle componenti precedenti, confermando che la forma complessiva deriva dalla sovrapposizione di contributi interpretabili.

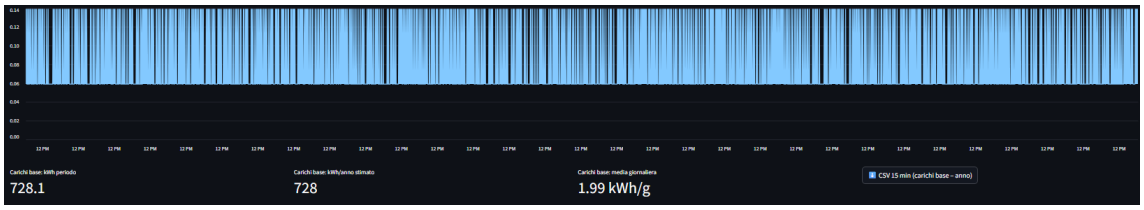

Figura 5.2: Esempio di profilo carichi base nel mese di gennaio.

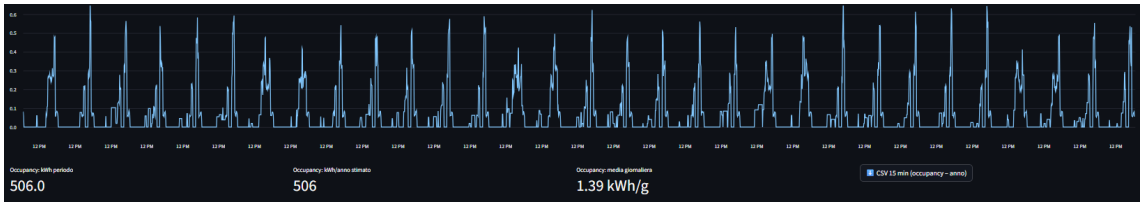

Figura 5.3: Esempio di profilo occupancy-driven nel mese di gennaio.

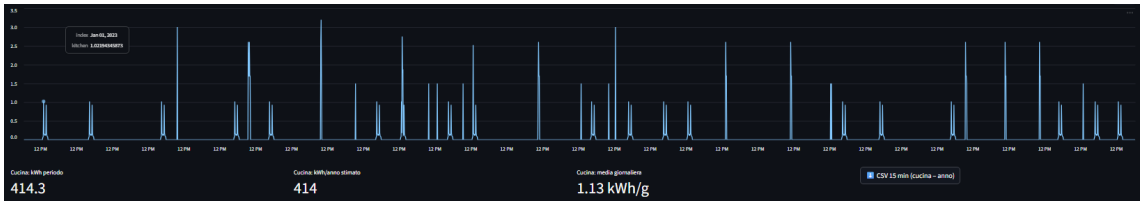

Figura 5.4: Esempio di profilo cucina (event-driven) nel mese di gennaio.

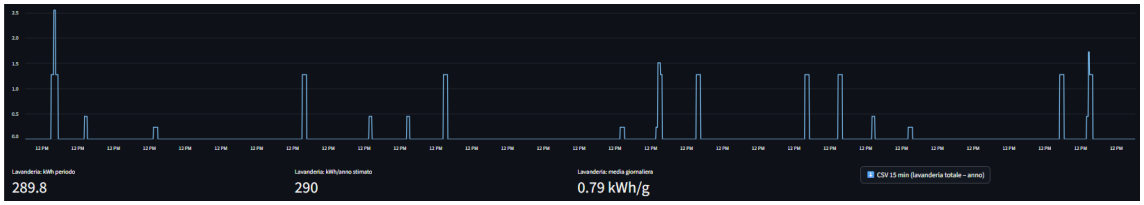

Figura 5.5: Esempio di profilo lavanderia nel mese di gennaio.

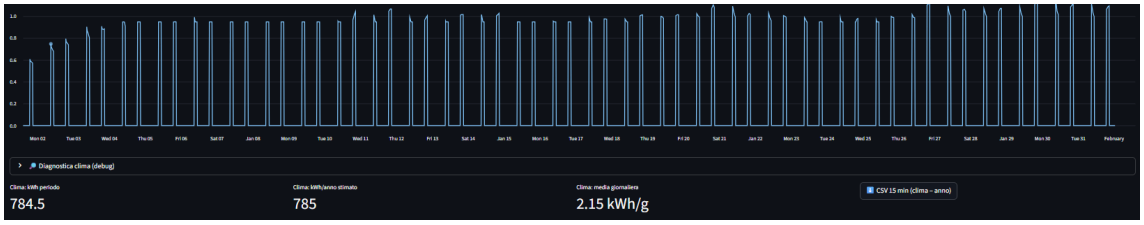

Figura 5.6: Esempio di profilo clima nel mese di gennaio.

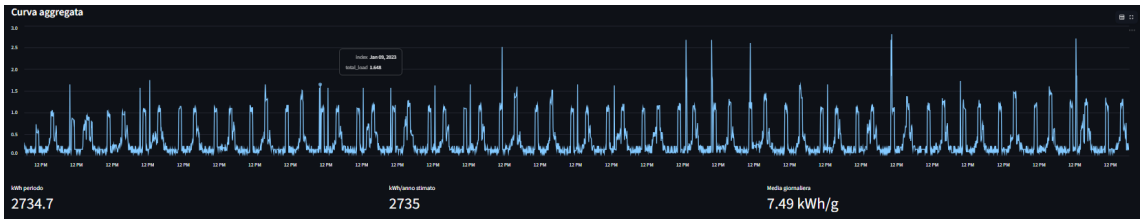

Figura 5.7: Esempio di profilo aggregato nel mese di gennaio.

Capitolo 6

Bilanciamento energetico: implementazione, flussi e energia condivisa

Il sottosistema di *bilanciamento energetico* costituisce lo stadio conclusivo della pipeline di simulazione: a valle della generazione delle curve di produzione (Capitolo 4) e delle curve di consumo (Capitolo 5), esso ricostruisce i flussi energetici dei singoli membri e della configurazione collettiva, con particolare attenzione all'energia condivisa su base oraria richiesta dalla regolazione dell'autoconsumo diffuso [3], [4].

Dal punto di vista implementativo, la pagina Streamlit dedicata (`3_Bilanciamento_CER.py`) orchestra la definizione e persistenza dello *scenario* (anagrafica membri, periodo e opzioni di simulazione), il caricamento dei profili di consumo/produzione e l'esecuzione del calcolo. La parte computazionale è concentrata nei moduli in `cer_core/bilanciamento/` (parsing, validazione, calcolo dei flussi e produzione degli output). Ogni esecuzione produce un *run* versionato, così da garantire riproducibilità e collegamento deterministico con la valutazione economica (Capitolo 7).

6.1 Ruolo del sottosistema e contratto dati nella pipeline

L'obiettivo del bilanciamento è trasformare profili di potenza dei membri (consumi a 15 minuti e produzione tipicamente oraria) in:

flussi energetici orari per ciascun membro ($E_{\mathrm{prel},i}(h)$, $E_{\mathrm{imm},i}(h)$, eventuali componenti di autoconsumo e accumulo); flussi aggregati di comunità ($E_{\mathrm{prel,CER}}(h)$,

$E_{\text{imm,CER}}(h)$) ed energia condivisa $E_{\text{cond}}(h)$; infine, indicatori energetici (KPI) a livello di membro e comunità..

I risultati sono persistiti in file tabellari con schema stabile (`members_hourly_long.csv` e `cer_hourly.csv`), così da costituire un *contratto dati*[1] verso la fase economica: la valutazione monetizza i prelievi, le immissioni e soprattutto l'energia condivisa $E_{\text{cond}}(h)$.

6.2 Scenario workspace e persistenza degli input

6.2.1 Struttura su filesystem per scenario e per membro

Per supportare confronti tra configurazioni e ripetibilità sperimentale, gli input del bilanciamento sono organizzati in uno *scenario workspace*[2] su filesystem. Ogni scenario è una cartella autonoma (ad es. `.../bilanciamento/scenarios/<scenario_name>/`) che contiene:

`scenario.json`: configurazione persistita (membri, periodo, opzioni produzione/BESS); `inputs/member_<id>/consumption_15min.csv`: profilo di consumo a 15 minuti (obbligatorio); `inputs/member_<id>/production_hourly.csv`: profilo di produzione orario (opzionale, per prosumer); `outputs/run_<run_id>/`: cartella risultati per singola esecuzione (run); infine, `runs_index.jsonl`[3] e `active_run.txt`: registro append-only delle esecuzioni e puntatore al run corrente..

Questa struttura separa nettamente *input* (scenario e CSV) da *output* (run), evitando che lo stato dell'interfaccia Streamlit diventi un requisito per la riproduzione dei risultati.

6.2.2 Modello dati dello scenario e migrazione legacy

Lo `scenario.json` rappresenta la sorgente di verità del bilanciamento e include:

elenco dei membri (identificativo, ruolo consumer/prosumer[4], moltiplicità `num`); configurazione della produzione (modalità `Totale` vs `Aree` con selezione delle colonne `Area_*`); configurazione del BESS (capacità, DoD, efficienze, SOC iniziale) quando

[1] Interfaccia dati tra moduli: schema e significato delle colonne sono mantenuti stabili così che componenti a valle possano consumare gli output senza dipendere dall'implementazione interna del bilanciamento.

[2] Directory di lavoro dedicata allo scenario, che raccoglie input e output della simulazione in una struttura di cartelle riproducibile.

[3] File in formato *JSON Lines* (JSONL): un record JSON per riga; è usato come registro append-only dei run.

[4] Nel seguito, *consumer* indica un'utenza con solo consumo; *prosumer* un'utenza che consuma e produce (ad es. tramite FV) e può quindi anche immettere energia in rete.

presente; infine, periodo di simulazione (se specificato) oppure parametri necessari per inferirlo..

6.3 Coerenza temporale e data quality

6.3.1 Normalizzazione timezone e convenzione UTC

Tutti i timestamp sono normalizzati a **UTC** in fase di parsing e persistenza. In particolare, timestamp *naive* vengono interpretati nel fuso orario dello scenario e poi convertiti a UTC; timestamp *timezone-aware*[5] vengono prima convertiti nel fuso orario dello scenario e infine normalizzati a UTC. Questa policy riduce ambiguità legate a cambi d'ora (DST) e rende confrontabili run e scenari.

6.3.2 Definizione del periodo e costruzione degli indici attesi (T15, TH)

Il periodo di simulazione può essere definito esplicitamente nello scenario oppure inferito dai dati di consumo a 15 minuti: il simulatore calcola t_0 come inizio giorno del primo campione e t_1 come fine giorno dell'ultimo campione (considerando la granularità a 15 minuti). Da $[t_0, t_1)$ vengono costruiti:

indice a 15 minuti T_{15} per il bilancio fisico (consumi, BESS); inoltre, indice orario T_H per aggregazione, energia condivisa e monetizzazione..

6.3.3 Validazioni e regole conservative sui dati mancanti

L'approccio adottato è conservativo: le serie vengono riallineate sugli indici attesi e sono accettati solo profili completi e regolari (monotonia, assenza di duplicati, passo temporale corretto). Eventuali buchi *interni* nella serie (missing tra due timestamp attesi) portano a un errore esplicito, evitando interpolazioni implicite che ridurrebbero la tracciabilità del risultato.

[5] Un timestamp *naive* non include informazione di fuso orario; un timestamp *timezone-aware* include un offset o una zona IANA. DST (*Daylight Saving Time*) indica il cambio d'ora legato all'ora legale.

6.4 Parsing e costruzione delle serie di consumo e produzione

6.4.1 Consumi a 15 minuti: formato, controlli e clipping

Con *clipping*[6] si indica il ritaglio del profilo di consumo al periodo $[t_0, t_1)$.

Ogni membro dispone di un profilo di consumo a 15 minuti in `consumption_15 min.csv` con almeno le colonne:

`timestamp`	istante (UTC dopo normalizzazione)
`total_load`	potenza assorbita [kW]

Il profilo viene validato e ritagliato sul periodo $[t_0, t_1)$, producendo una serie regolare su T_{15}.

6.4.2 Produzione oraria: modalità Totale vs Aree e riallineamento su T15

La produzione, quando disponibile, è fornita su base oraria in `production_hourly.csv` con colonna temporale `time` e:

modalità `Totale`: colonna `Totale`; inoltre, modalità `Aree`: somma di sottocolonne `Area_*` selezionate nello scenario..

La serie oraria viene validata e riallineata a T_H; per il calcolo fisico a 15 minuti viene poi trasformata su T_{15} tramite *zero-order hold*[7] (replica del valore orario sui quattro intervalli da 15 minuti).

6.5 Bilancio energetico del membro (senza accumulo)

6.5.1 Notazione, unità e conversione kW→kWh

Nel seguito, $P_{\text{load},i}(t)$ indica la potenza di carico del membro i al passo $t \in T_{15}$, mentre $P_{\text{pv}}(t)$ indica la potenza di produzione locale (eventualmente aggregata). Le energie sul passo sono ottenute tramite:

$$E(t) = P(t)\,\Delta t, \qquad \Delta t = 0.25 \text{ h}. \tag{6.1}$$

[6] Ritaglio della serie al periodo di simulazione $[t_0, t_1)$, scartando i campioni esterni e garantendo coerenza con gli indici temporali attesi.

[7] Tecnica di ri-campionamento in cui il segnale viene mantenuto costante tra due campioni (interpolazione a tratti costanti).

6.5.2 Definizione di autoconsumo, prelievo e immissione

Al passo t l'energia autoconsumata è:

$$E_{\mathrm{aut},i}(t) = \min\big(E_{\mathrm{load},i}(t), E_{\mathrm{pv}}(t)\big). \tag{6.2}$$

Le componenti di scambio con rete sono quindi:

$$E_{\mathrm{prel},i}(t) = E_{\mathrm{load},i}(t) - E_{\mathrm{aut},i}(t), \qquad E_{\mathrm{imm},i}(t) = E_{\mathrm{pv}}(t) - E_{\mathrm{aut},i}(t). \tag{6.3}$$

6.5.3 Aggregazione oraria dei flussi

Poiché la regolazione dell'energia condivisa opera su base oraria [3], [4], i flussi vengono aggregati su T_H tramite somma degli intervalli a 15 minuti:

$$E_{\bullet,i}(h) = \sum_{t \in h} E_{\bullet,i}(t), \tag{6.4}$$

dove $\bullet$ indica una qualunque componente energetica (prelievo, immissione, autoconsumo, ecc.).

6.6 Estensione con accumulo elettrochimico (BESS) in logica RSE-like

6.6.1 Parametri, vincoli SOC e efficienze (half-cycle)

Il BESS è modellato a 15 minuti mediante stato di carica (SOC[8]) espresso in kWh, con capacità C e profondità di scarica massima DoD[9]. I limiti di SOC sono:

$$SOC_{\min} = C\,(1 - DoD), \qquad SOC_{\max} = C. \tag{6.5}$$

Data un'efficienza round-trip[10] η_{rt}, il modello utilizza una half-cycle efficiency:

$$\eta_{\mathrm{hc}} = \sqrt{\eta_{\mathrm{rt}}}, \tag{6.6}$$

coerente con una scomposizione simmetrica delle perdite tra fase di carica e scarica.

[8] *State of Charge*: energia disponibile nell'accumulo rispetto alla capacità nominale; in questo modello il SOC è espresso in kWh.

[9] *Depth of Discharge*: quota massima di capacità che può essere scaricata; determina il limite inferiore $SOC_{\min}$.

[10] L'efficienza *round-trip* misura la resa complessiva del ciclo carica+scarica; la *half-cycle efficiency* rappresenta la resa per singola fase (carica o scarica).

6.6.2 Step di simulazione a 15 minuti e aggiornamento del SOC

Si definisce l'energia netta ai morsetti in assenza di BESS:

$$E_{\text{term}}(t) = E_{\text{pv}}(t) - E_{\text{load},i}(t). \tag{6.7}$$

La logica operativa segue una priorità locale: surplus di produzione → carica, deficit → scarica, residuo → rete, rispettando i vincoli $SOC_{\min}$ e $SOC_{\max}$. L'evoluzione dello stato di carica è:

$$SOC(t + \Delta t) = SOC(t) + E_{\text{bess}}(t), \tag{6.8}$$

dove $E_{\text{bess}}(t)$ è positiva in carica e negativa in scarica (in kWh sul passo).

Il bilancio con rete dopo l'azione della batteria diventa:

$$E_{\text{grid},i}(t) = E_{\text{pv}}(t) - E_{\text{load},i}(t) - E_{\text{bess}}(t), \tag{6.9}$$

con:

$$E_{\text{imm},i}(t) = \max\big(0, E_{\text{grid},i}(t)\big), \qquad E_{\text{prel},i}(t) = -\min\big(0, E_{\text{grid},i}(t)\big). \tag{6.10}$$

6.6.3 Discussione del livello di astrazione (energy-based) e implicazioni

Il modello implementato è *energy-based*[11]: la dinamica è espressa in termini di energia scambiata sul passo Δt e vincoli di SOC, senza introdurre esplicitamente limiti di potenza di carica/scarica (kW) o vincoli inverter[12]. Questa scelta semplifica l'analisi scenario-based e mantiene trasparenza della pipeline, ma può sovrastimare la capacità del BESS di seguire picchi rapidi: tale limite viene considerato nella discussione finale (Sezione 6.10).

[11] Approccio in cui la dinamica è modellata in energia (kWh) sul passo Δt, con vincoli sullo stato (SOC), invece che tramite un modello in potenza (kW) con limiti dinamici di carica/scarica.

[12] Vincoli imposti dal convertitore/inverter (ad es. potenza massima e limiti di corrente) che possono ridurre la capacità effettiva di carica/scarica rispetto a un modello puramente energetico.

6.7 Aggregazione di comunità ed energia condivisa

6.7.1 Somma dei flussi e definizione dell'energia condivisa oraria

Indicando con $E_{\text{prel},i}(h)$ ed $E_{\text{imm},i}(h)$ i flussi orari dei membri, i flussi aggregati della comunità sono:

$$E_{\text{prel,CER}}(h) = \sum_i E_{\text{prel},i}(h), \qquad E_{\text{imm,CER}}(h) = \sum_i E_{\text{imm},i}(h). \tag{6.11}$$

L'energia condivisa oraria è definita come:

$$E_{\text{cond}}(h) = \min\big(E_{\text{prel,CER}}(h), E_{\text{imm,CER}}(h)\big), \tag{6.12}$$

coerentemente con la definizione regolatoria su base oraria [3], [4].

6.7.2 Residui verso rete (import/export) e controlli di coerenza

I residui verso rete, una volta "estratta" l'energia condivisa, sono:

$$E_{\text{imp,CER}}(h) = E_{\text{prel,CER}}(h) - E_{\text{cond}}(h), \qquad E_{\text{exp,CER}}(h) = E_{\text{imm,CER}}(h) - E_{\text{cond}}(h). \tag{6.13}$$

Per costruzione, valgono le disuguaglianze $E_{\text{cond}}(h) \leq E_{\text{prel,CER}}(h)$ e $E_{\text{cond}}(h) \leq E_{\text{imm,CER}}(h)$, utili come controllo di plausibilità.

6.8 Output del bilanciamento e versionamento dei run

6.8.1 Schema degli output (CER, membri long, summary, metadati)

Ogni esecuzione produce output strutturati e riutilizzabili:

`cer_hourly.csv`: profilo orario aggregato (include $E_{\text{cond}}(h)$); `members_hourly_long.csv`: profili orari dei membri in formato long[13] (chiave `member_id`); `memb`

[13] Rappresentazione *long* (o *tidy*): una riga per combinazione (timestamp, membro, variabile), utile per filtri e aggregazioni.

`ers_summary.csv`, `cer_summary.csv`: KPI energetici; infine, `period_meta.csv`: metadati sul periodo e sulla normalizzazione temporale..

Questi file costituiscono l'output minimo necessario per monetizzare il comportamento della CER nel Capitolo 7.

6.8.2 Registro append-only, run attivo e snapshot di configurazione

Dal punto di vista della riproducibilità, l'esecuzione della simulazione crea una cartella dedicata `outputs/run_<run_id>/` (con `run_id` tipicamente basato su timestamp UTC) contenente gli output e uno *snapshot*[14] della configurazione effettivamente usata (ad es. `run_config.json`) oltre a metadati di run (ad es. `run_meta.json`).

A supporto della tracciabilità tra esecuzioni successive, lo scenario mantiene inoltre un registro dei run energetici (file `runs_index.jsonl`) e un puntatore al run attivo (file `active_run.txt`). Il registro viene aggiornato in modalità append-only e associa a ciascun run il relativo identificativo, i percorsi dei principali output e un fingerprint[15] dei file di configurazione. Tale meccanismo consente di ricaricare in modo deterministico l'ultimo run validato e di confrontare run diversi senza dipendere dallo stato volatile dell'interfaccia Streamlit; inoltre abilita l'aggancio diretto della valutazione economica al run energetico selezionato (Capitolo 7).

6.9 Confronto mirato con CACER_Simulator (RSE)

Le logiche principali di bilanciamento (definizione dell'energia condivisa oraria e dinamica di accumulo in termini di SOC e vincoli) sono coerenti con l'impostazione del simulatore RSE per CACER [30]. La differenza principale risiede nell'architettura: l'integrazione in un'applicazione interattiva consente gestione di scenari, controlli di qualità del dato, persistenza su filesystem e versionamento dei run, rendendo più agevole l'analisi comparativa tra configurazioni (ad esempio variazione della potenza FV, introduzione del BESS o variazioni nel mix di utenze).

6.10 Limiti noti ed estensioni

I principali limiti (e potenziali estensioni) del sottosistema di bilanciamento includono:

[14] Copia "congelata" della configurazione (e, se rilevante, dei parametri derivati) effettivamente usata in un run, per poter ricostruire a posteriori le condizioni di esecuzione.

[15] Impronta deterministica (hash) calcolata a partire da file/parametri di input: consente di verificare che configurazione e dati non siano cambiati tra esecuzioni.

Vincoli di potenza del BESS: l'assenza di limiti kW può sovrastimare l'azione su picchi; un'estensione naturale è introdurre $P_{\max}^{\mathrm{ch}}$ e $P_{\max}^{\mathrm{dis}}$ e vincoli inverter.; **Rappresentatività temporale**: l'annualizzazione o l'uso di finestre temporali parziali può introdurre bias stagionali; è utile supportare periodi multi-anno o pesi stagionali.

Capitolo 7

Valutazione economica dei progetti CER e ottimizzazione del BESS

Questo capitolo descrive il modulo di valutazione economico-finanziaria del simulatore e la logica di dimensionamento dell'accumulo elettrochimico (BESS) in funzione di indicatori finanziari. Il sottosistema di bilanciamento energetico (Capitolo 6) restituisce le serie orarie degli scambi di energia a livello di membro e di comunità; il modulo economico traduce tali grandezze in corrispettivi monetari (costi, ricavi e incentivi) e costruisce un modello di cassa multi-anno, dal quale si derivano KPI tipici di progetto (NPV, IRR e payback).

Nel simulatore, la valutazione economica è intesa come un passaggio strettamente dipendente dal run energetico[1]: ogni valutazione memorizza uno snapshot[2] completo delle assunzioni utilizzate (*assumption pack*[3]) e un registro di run, in modo da consentire confronti riproducibili tra configurazioni differenti (potenza FV, mix di utenze, presenza/assenza BESS, parametri tariffari e fiscali).

7.1 Dati in ingresso e collegamento al run energetico

La valutazione economica opera a valle di un run energetico validato. In termini implementativi, la pagina Streamlit dedicata (`4_Valutazione_Economica.py`) carica

[1]Per "run" si intende una singola esecuzione completa del simulatore (energetica o economica) con un insieme di input fissati, identificata e tracciata tramite output versionati (ad es. cartella timestamp e metadati) per garantire riproducibilità.

[2]Con "snapshot" si indica una copia immutabile (al momento del run) delle impostazioni e dei parametri usati, salvata insieme agli output per poter ricostruire ex post la configurazione di calcolo.

[3]L'*assumption pack* ("pacchetto di assunzioni") raccoglie in modo strutturato tariffe, profili di prezzo/incentivo e parametri fiscali/tecnici necessari al calcolo economico.

dal run attivo di bilanciamento due dataset canonici[4]: (i) il profilo orario della comunità (`cer_hourly.csv`), contenente in particolare l'energia condivisa $E_{\text{cond}}(h)$; (ii) il profilo orario dei membri in formato long (`members_hourly_long.csv`), contenente per ciascun membro i le energie di prelievo $E_{\text{prel},i}(h)$ e immissione $E_{\text{imm},i}(h)$, oltre alle componenti di autoconsumo e agli eventuali indicatori associati alla batteria.

A livello concettuale, il contratto dati tra bilanciamento ed economia è definito dalle seguenti grandezze orarie:

$$E_{\text{prel},i}(h), \quad E_{\text{imm},i}(h), \quad E_{\text{aut},i}(h), \quad E_{\text{cond}}(h), \qquad h \in \mathcal{H}, \tag{7.1}$$

In particolare, $E_{\text{aut},i}(h)$ rappresenta l'energia autoconsumata dal membro (da produzione locale e/o da scarica BESS, a seconda della configurazione) ed è utilizzata per monetizzare il risparmio in bolletta nell'ottica di cassa descritta in Sezione 7.5. dove $\mathcal{H}$ è l'insieme delle ore simulate (timezone UTC, intervalli left-inclusive/right-exclusive). Le grandezze individuali derivano dalla simulazione a 15 min e dalla successiva aggregazione oraria (Sezioni 6.5–6.6). L'energia condivisa $E_{\text{cond}}(h)$ è definita come il minimo tra la somma delle immissioni e la somma dei prelievi in comunità (Sezione 6.7).

7.2 Assunzioni economiche e profili di prezzo/incentivo

La valutazione economica richiede, oltre ai flussi energetici calcolati nel run tecnico (energia autoconsumata, condivisa e scambiata con la rete), un insieme di assunzioni che descrivono i prezzi di acquisto e vendita dell'energia e i profili di incentivo. Nel simulatore, tali assunzioni sono gestite come *assumption pack* composto da: (i) parametri tabellari (politiche, tariffe, fattori) e (ii) profili temporali orari opzionali ($PZO(h)$), che possono essere fissati, caricati dall'utente o derivati in modo deterministico da altre variabili.

7.2.1 Profili temporali: convenzioni, unità e validazione

Tutti i profili economici utilizzati dal modello sono serie orarie indicizzate da timestamp e misurate in €/kWh. Operativamente, il simulatore assume che i profili siano allineati alla discretizzazione temporale del run energetico; in particolare, un profilo orario deve coprire integralmente l'intervallo temporale della simulazione.

[4]Nel presente lavoro, per "dataset canonici" si intendono file di scambio in formato e nomenclatura standardizzati, prodotti dal modulo di bilanciamento e consumati dal modulo economico. Essi costituiscono la parte operativa del "contratto dati", ossia la specifica delle variabili richieste (colonne, unità e convenzioni temporali) all'interfaccia tra i due moduli.

Qualora un profilo richiesto non sia disponibile o non copra l'intero intervallo, il run economico non procede, evitando interpolazioni implicite che ridurrebbero la tracciabilità dei risultati.

7.2.2 Prezzo zonale orario: definizione e modalità di acquisizione

Nel presente lavoro il *Prezzo Zonale Orario* (PZO) è trattato come input esogeno del modulo economico, rappresentato da una serie oraria:

$$PZO(h) \quad [€/\text{kWh}], \tag{7.2}$$

dove h denota l'ora di simulazione.

Il simulatore non effettua il download automatico del PZO: l'utente può (i) caricare un profilo storico/previsto ottenuto da fonti esterne (ad es. esportazioni rese disponibili dal GME, nel rispetto di credenziali e condizioni d'uso dei servizi), oppure (ii) generare un profilo sintetico all'interno dell'interfaccia (profilo casuale riproducibile mediante seme). Questa scelta progettuale separa chiaramente la catena di acquisizione dati (fuori dallo scope del modello) dal calcolo economico, favorendo riproducibilità e auditabilità.

Formato del file CSV per il profilo PZO

Il profilo PZO è caricato come file CSV con una colonna temporale e una colonna di prezzo. Il formato consigliato è il seguente:

- `time`: timestamp orario (convertibile in `datetime`);
- `pzo_eur_kwh`: valore del prezzo in €/kWh.

Generazione sintetica del PZO

In alternativa al caricamento, il simulatore consente la generazione di un profilo orario sintetico $PZO(h)$ mediante parametri statistici (media, deviazione standard) e un seme casuale. Il profilo risultante è non-negativo e riproducibile a parità di parametri e seme; tale opzione è utile per analisi di sensibilità e test di robustezza del modello.

7.2.3 Prezzo di acquisto dell'energia

Il prezzo di acquisto dell'energia prelevata dalla rete per l'utente i è modellato come profilo orario $p_i^{buy}(h)$ in €/kWh. Nel simulatore sono implementate tre modalità.

Prezzo fisso. Il prezzo è costante nel tempo:

$$p_i^{buy}(h) = \bar{p}_i^{buy}. \tag{7.3}$$

Prezzo in tre fasce (F1/F2/F3). Il prezzo assume tre livelli in funzione della fascia oraria:

$$p_i^{buy}(h) \in \{p_{F1}, p_{F2}, p_{F3}\}, \tag{7.4}$$

dove l'assegnazione della fascia è effettuata secondo la classificazione oraria su calendario locale.

Prezzo indicizzato a PZO con spread. Il prezzo è descritto come funzione affine del PZO:

$$p_i^{buy}(h) = m_i^{buy} \cdot PZO(h) + s_i^{buy}, \tag{7.5}$$

dove m_i^{buy} è un moltiplicatore (adimensionale) e s_i^{buy} uno spread in €/kWh. Questa opzione permette di rappresentare in modo parametrico una fornitura indicizzata a mercato con correzione contrattuale.

Lo stesso profilo $p_i^{buy}(h)$ viene inoltre impiegato per valorizzare, in ottica DCF[5], il beneficio economico dell'energia autoconsumata (risparmio bolletta), come descritto in Sezioni 7.3 e 7.5.

7.2.4 Prezzo di vendita dell'energia

Il prezzo di vendita dell'energia immessa in rete dall'utente i è modellato come profilo orario $p_i^{sell}(h)$ in €/kWh. Nel simulatore sono supportate le seguenti modalità.

Prezzo fisso.

$$p_i^{sell}(h) = \bar{p}_i^{sell}. \tag{7.6}$$

Prezzo indicizzato a PZO.

$$p_i^{sell}(h) = m_i^{sell} \cdot PZO(h). \tag{7.7}$$

[5]DCF (*Discounted Cash Flow*): modello di valutazione che attualizza i flussi di cassa futuri a un tasso di sconto r per confrontarli su base equivalente al tempo zero.

Prezzo indicizzato a PZO con fee.

$$p_i^{sell}(h) = m_i^{sell} \cdot PZO(h) - f_i^{sell}, \tag{7.8}$$

dove f_i^{sell} è una fee in €/kWh. Tale formulazione consente di rappresentare costi di sbilanciamento/gestione o trattenute contrattuali.

7.2.5 Profilo di incentivo TIP

Il simulatore rappresenta l'incentivo unitario (TIP) come profilo orario $TIP(h)$ in €/kWh. In funzione della modalità selezionata, il profilo può essere fissato, caricato o derivato da $PZO(h)$.

TIP fisso.

$$TIP(h) = \overline{TIP}. \tag{7.9}$$

TIP come funzione semplice di PZO. Per analisi parametriche, il simulatore include una modalità deterministica in cui $TIP(h)$ dipende da $PZO(h)$ tramite un termine base e un termine additivo limitato da un cap:

$$TIP(h) = TIP_{base} + \min\Big(cap_{max}, \max\big(0, cap_{ref} - PZO(h)\big)\Big). \tag{7.10}$$

Questa formulazione ha finalità modellistica (sensibilità rispetto al prezzo di mercato) e non intende sostituire una determinazione normativa.

TIP in modalità RSE-like. Il simulatore implementa inoltre una modalità "RSE-like" coerente con i riferimenti regolatori assunti nel capitolo. In tale modalità, il calcolo è espresso in €/MWh e poi convertito in €/kWh. Si definisce:

$$PZO_{MWh}(h) = 1000 \cdot PZO(h). \tag{7.11}$$

Si introduce quindi una componente variabile legata al differenziale rispetto a una soglia (in €/MWh):

$$var(h) = \max\big(0, 180 - PZO_{MWh}(h)\big). \tag{7.12}$$

Il modello utilizza parametri (tp_{base}, cap_{var}) determinati per scaglioni di potenza dell'impianto, e un addendo fisso dipendente dalla macro-area ("fattore zonale"). Inoltre, è prevista una riduzione legata all'intensità di grant (ad es. PNRR), modellata

tramite un fattore moltiplicativo. Indicando con F_{zone} l'addendo macro-areale e con f_{grant} il fattore di riduzione, la tariffa (in €/MWh) è ricondotta alla seguente struttura:

$$TIP_{MWh}(h) = \Big(\min \big(cap_{var},\, tp_{base} + var(h) \big) + F_{zone} \Big) \cdot (1 - f_{grant}), \tag{7.13}$$

e quindi:

$$TIP(h) = \frac{TIP_{MWh}(h)}{1000}. \tag{7.14}$$

7.2.6 Profilo di restituzione componenti TIAD

Analogamente, il simulatore descrive la componente TIAD come profilo orario $TIAD(h)$ in €/kWh. Sono disponibili le modalità seguenti.

TIAD fisso.

$$TIAD(h) = \overline{TIAD}. \tag{7.15}$$

TIAD in modalità RSE-like (ARERA-like). In questa modalità il calcolo dipende dal tipo di configurazione (CER, AUC o assenza di CACER). Si utilizzano parametri espressi in €/MWh (ad es. `TRASe`, `BTAU`), che vengono convertiti in €/kWh dividendo per 1000.

Nel caso CER, $TIAD(h)$ è modellato costante:

$$TIAD(h) = TRASe. \tag{7.16}$$

Nel caso AUC, $TIAD(h)$ include una componente proporzionale al PZO pesata sulle quote BT/MT:

$$TIAD(h) = TRASe + BTAU + \Big(share_{BT} \cdot Cpr_{BT} + share_{MT} \cdot Cpr_{MT} \Big) \cdot PZO(h), \tag{7.17}$$

dove $share_{MT} = 1 - share_{BT}$. Qualora $share_{BT}$ non sia specificato, il simulatore tenta una stima a partire dal livello di tensione degli utenti; in assenza di informazione, viene assunto $share_{BT} = 1$.

7.2.7 Detrazione fiscale (Bonus ristrutturazioni): parametrizzazione

Nel simulatore la detrazione fiscale associata agli interventi (ad es. impianto FV, BESS e generazione eolica, ove ammessi) è modellata come un beneficio di cassa

aggiuntivo, ripartito in quote annuali costanti per un numero prefissato di anni. Tale scelta permette di rappresentare in modo trasparente l'effetto della detrazione sugli indicatori DCF senza alterare il conto economico (PnL)[6] del progetto.

Indicando con $CAPEX_i^{PV}(0)$, $CAPEX_i^{BESS}(0)$ e $CAPEX_i^{EOL}(0)$ gli investimenti sostenuti al tempo zero, il simulatore definisce la spesa eleggibile:

$$CAPEX_i^{\text{elig}} = CAPEX_i^{PV}(0) + CAPEX_i^{BESS}(0) + CAPEX_i^{EOL}(0), \tag{7.18}$$

a cui applica un massimale parametrico CAP_{det} (espresso in € e tipicamente riferito a unità immobiliare) e un'aliquota di detrazione $\tau_{det} \in [0, 1]$:

$$Det_i^{\text{tot}} = \tau_{det} \cdot \min\Big(CAPEX_i^{\text{elig}},\, CAP_{det}\Big)\,. \tag{7.19}$$

Il credito complessivo Det_i^{tot} è quindi ripartito su N_{det} anni (default 10) in quote costanti:

$$D_i(y) = \begin{cases} \dfrac{Det_i^{\text{tot}}}{N_{det}} & \text{se } 1 \le y \le N_{det}, \\ 0 & \text{altrimenti.} \end{cases} \tag{7.20}$$

Nella configurazione attuale, la simulazione assume capienza fiscale sufficiente a utilizzare integralmente la quota annua; eventuali estensioni possono introdurre un vincolo di capienza (ad es. $D_i(y) = \min(D_i(y), \text{IRPEF_cap}_i(y))$).

7.2.8 Osservazioni operative e limiti

Le modalità sopra descritte implementano esplicitamente i driver considerati nel simulatore (PZO, spread/fee, scaglioni di potenza, macro-area, intensità di grant, tipo CACER e ripartizione BT/MT). Eventuali ulteriori fattori (ad es. condizioni contrattuali complesse, indicatori macroeconomici o tariffe dinamiche non riconducibili a PZO) richiedono un'estensione del modello, da documentare con coerenza rispetto al codice sorgente e ai dataset impiegati.

7.3 Costi di acquisto, ricavi da vendita e incentivi

A partire dalle serie energetiche orarie, il simulatore calcola i driver monetari orari. Il costo di acquisto dell'energia per il membro i è:

$$C_i^{\text{buy}}(h) = E_{\text{prel},i}(h)\, p_i^{\text{buy}}(h), \tag{7.21}$$

[6] PnL (*Profit and Loss statement*): rendiconto economico che riassume ricavi e costi (e quindi l'utile/perdita) nel periodo considerato.

In aggiunta, si definisce il *risparmio da autoconsumo* (beneficio economico associato all'energia non acquistata dalla rete):

$$S_i^{\text{aut}}(h) = E_{\text{aut},i}(h)\, p_i^{\text{buy}}(h). \tag{7.22}$$

Tale quantità viene utilizzata nel modello di cassa DCF (Sezione 7.5) come incremento dei flussi, mentre il costo di acquisto $C_i^{\text{buy}}(h)$ continua a essere rendicontato nel PnL.

Il ricavo associato alle immissioni (se la vendita è abilitata per l'utente) è:

$$R_i^{\text{sell}}(h) = E_{\text{imm},i}(h)\, p_i^{\text{sell}}(h). \tag{7.23}$$

Gli incentivi sono calcolati sull'energia condivisa a livello di comunità. Definendo:

$$R_{\text{tot}}^{TIP}(h) = E_{\text{cond}}(h)\, TIP(h), \qquad R_{\text{tot}}^{TIAD}(h) = E_{\text{cond}}(h)\, TIAD(h), \tag{7.24}$$

si applica una regola di ripartizione parametrica tra consumatori e produttori basata sul coefficiente $\alpha_c \in [0, 1]$:

$$R_c^{TIP}(h) = \alpha_c\, R_{\text{tot}}^{TIP}(h), \qquad R_p^{TIP}(h) = (1 - \alpha_c)\, R_{\text{tot}}^{TIP}(h), \tag{7.25}$$

e in modo analogo per $R_c^{TIAD}(h)$ e $R_p^{TIAD}(h)$. In questa formulazione, α_c rappresenta l'input di policy definito dall'utente nella UI (ad esempio 0.45 per allocare il 45% ai consumatori e il restante 55% ai produttori).

7.4 Allocazione degli incentivi ai membri

L'allocazione degli incentivi ai singoli membri è effettuata su base oraria e proporzionale ai flussi di prelievo e immissione. Si definiscono quindi i pesi:

$$w_i^{\text{prel}}(h) = \frac{E_{\text{prel},i}(h)}{\sum_j E_{\text{prel},j}(h)}, \qquad w_i^{\text{imm}}(h) = \frac{E_{\text{imm},i}(h)}{\sum_j E_{\text{imm},j}(h)}, \tag{7.26}$$

con convenzione $w = 0$ quando il denominatore è nullo. Il ricavo incentivante attribuito al membro i diventa:

$$R_i^{TIP}(h) = R_c^{TIP}(h)\, w_i^{\text{prel}}(h) + R_p^{TIP}(h)\, w_i^{\text{imm}}(h), \tag{7.27}$$

$$R_i^{TIAD}(h) = R_c^{TIAD}(h)\, w_i^{\text{prel}}(h) + R_p^{TIAD}(h)\, w_i^{\text{imm}}(h). \tag{7.28}$$

Questa scelta garantisce coerenza con la definizione di energia condivisa: la

componente "consumatori" remunera i membri in funzione del loro contributo ai prelievi, mentre la componente "produttori" remunera i membri in funzione del loro contributo alle immissioni. In termini implementativi, le quantità $w_i^{\text{prel}}(h)$ e $w_i^{\text{imm}}(h)$ sono calcolate con divisione sicura (*safe division*) per evitare instabilità numeriche quando, in una certa ora, la comunità non presenta prelievi o non presenta immissioni.

7.5 Bilancio economico annuale e modello DCF

I driver orari sono aggregati a livello annuale per costruire un conto economico e un modello di cassa multi-anno. Indicando con year = 1 l'anno rappresentato dai profili energetici (tipicamente un anno completo), si definiscono i totali:

$$C_i^{\text{buy}}(1) = \sum_{h \in \mathcal{H}} C_i^{\text{buy}}(h), \qquad R_i^{\text{sell}}(1) = \sum_{h \in \mathcal{H}} R_i^{\text{sell}}(h), \tag{7.29}$$

$$S_i^{\text{aut}}(1) = \sum_{h \in \mathcal{H}} S_i^{\text{aut}}(h). \tag{7.30}$$

$$R_i^{TIP}(1) = \sum_{h \in \mathcal{H}} R_i^{TIP}(h), \qquad R_i^{TIAD}(1) = \sum_{h \in \mathcal{H}} R_i^{TIAD}(h). \tag{7.31}$$

Nel caso in cui il run energetico copra un sotto-periodo dell'anno (ad esempio un trimestre), il simulatore può normalizzare i totali annuali tramite un fattore di scaling basato sul numero di ore simulate. Tale opzione consente valutazioni preliminari a partire da dataset parziali, pur mantenendo esplicita la distinzione tra dati osservati e annualizzazione.

A partire dai totali annuali, il simulatore costruisce per ciascun membro un conto economico semplificato, includendo OPEX[7], eventuali ammortamenti e una componente fiscale parametrica dipendente dalla classe di utenza (famiglie/imprese) con possibilità di override per singolo membro. Inoltre, il modello include costi di investimento (CAPEX[8]) al tempo zero e costi di sostituzione (*replacement*) in anni specifici, coerentemente con le vite utili dichiarate per gli asset (FV e BESS). In questa implementazione si distinguono: (i) il conto economico (PnL), che include tra i costi operativi il costo di acquisto dell'energia dalla rete; (ii) il flusso di cassa utilizzato per la valutazione DCF, costruito a partire dall'utile netto ma con aggiustamenti *incrementali* coerenti con l'ottica del presente lavoro. In particolare, il costo di acquisto dell'energia viene mantenuto nel PnL per fini di rendicontazione, ma è

[7]OPEX (*Operating Expenditure*): costi operativi ricorrenti associati all'esercizio e alla manutenzione degli asset, sostenuti lungo la vita del progetto.

[8]CAPEX (*Capital Expenditure*): spese di investimento iniziali per l'acquisto/realizzazione degli asset (ad es. FV e BESS), tipicamente sostenute al tempo zero.

escluso dal cash flow di valutazione mediante un add-back[9]; inoltre, si include (a) il risparmio monetario legato all'autoconsumo e (b) l'eventuale beneficio di detrazione fiscale.

In forma compatta, il flusso di cassa DCF è quindi espresso come:

$$\begin{aligned} CF_i(0) &= -CAPEX_i(0), \\ CF_i(y) &= NI_i(y) + DA_i(y) - CAPEX_i^{\text{repl}}(y) + C_i^{\text{buy}}(y) + S_i^{\text{aut}}(y) + D_i(y), \end{aligned} \tag{7.32}$$

dove $NI_i(y)$ è l'utile netto del PnL, $DA_i(y)$ gli ammortamenti, $CAPEX_i^{\text{repl}}(y)$ la spesa per sostituzioni nell'anno y, $C_i^{\text{buy}}(y)$ il costo annuo di acquisto energia (add-back nel DCF), $S_i^{\text{aut}}(y)$ il risparmio annuo da autoconsumo e $D_i(y)$ la quota annua di detrazione fiscale (Sezione 7.2.7).

Rispetto a una definizione puramente contabile del cash flow (senza add-back e senza componenti aggiuntive), la presente formulazione tende a incrementare i flussi dei primi anni e, di conseguenza, i KPI di progetto (NPV, IRR e payback). Pertanto, i risultati ottenuti con questa impostazione non sono direttamente confrontabili con quelli derivanti da un DCF che includa il costo di acquisto energia come uscita di cassa.

A partire dai flussi di cassa si definiscono i principali indicatori di valutazione (VAN/NPV, TIR/IRR e payback). In particolare, il VAN (o NPV) del membro i è dato da:

$$NPV_i = \sum_{y=0}^{Y} \frac{CF_i(y)}{(1+r)^y}, \tag{7.33}$$

dove r è il tasso di sconto e Y l'orizzonte di valutazione. La TIR (o IRR) è definita come il valore $r^\star$ che annulla il VAN:

$$0 = \sum_{y=0}^{Y} \frac{CF_i(y)}{(1+r^\star)^y}. \tag{7.34}$$

Il payback è calcolato come il primo anno in cui la somma cumulata dei flussi di cassa diventa non negativa. In forma esplicita, introducendo i cumulati semplice e scontato

$$\text{CumCF}_i(y) = \sum_{\tau=0}^{y} CF_i(\tau), \qquad \text{CumDCF}_i(y) = \sum_{\tau=0}^{y} \frac{CF_i(\tau)}{(1+r)^\tau}, \tag{7.35}$$

[9]Nel presente contesto, per *add-back* si intende la riaggiunta nel cash flow di valutazione di una voce di costo già contabilizzata nel PnL, cosìda neutralizzarne l'impatto sul flusso di cassa (coerentemente con l'ottica incrementale adottata).

si definiscono, rispettivamente, il payback semplice e quello scontato come:

$$PB_i = \min\{y \in [0, Y] : \mathrm{CumCF}_i(y) \geq 0\}, \qquad DPB_i = \min\{y \in [0, Y] : \mathrm{CumDCF}_i(y) \geq 0\}. \tag{7.36}$$

7.6 Output economici e tracciabilità

Ogni valutazione economica produce una cartella di output dedicata (timestamp) nella directory di sessione `economics/outputs/`. Il folder contiene: (i) le tabelle di risultato (`pnl_by_member.csv`, `cashflow_by_member.csv`, `kpis_by_member.csv`, e le corrispondenti versioni aggregate); (ii) un file `manifest.json` che elenca gli artefatti presenti e facilita la discovery programmatica dei risultati; (iii) lo snapshot completo delle assunzioni economiche, salvato nella stessa cartella a corredo dei risultati.

In particolare, la tabella `cashflow_by_member.csv` include esplicitamente le componenti aggiuntive introdotte nell'ottica DCF, tra cui il risparmio da autoconsumo (`savings_autocons`) e la quota annua di detrazione fiscale (`tax_detraction`), a supporto della tracciabilità e del confronto tra scenari.

In modo analogo al bilanciamento, anche per l'economia è mantenuto un registro a livello di sessione (`economics/runs_index.jsonl`) e un puntatore al run economico attivo (`economics/active_run.txt`). Il record di run include riferimenti ai file principali e fingerprint degli input (scenario energetico, run energetico selezionato e assumption pack economico). Questo meccanismo rende esplicita la catena di dipendenze tra energia ed economia e consente confronti riproducibili tra diverse ipotesi tariffarie, fiscali o di investimento.

7.7 Ottimizzazione del dimensionamento BESS in funzione dell'NPV

Nel simulatore, l'ottimizzazione del BESS non è implementata come uno *sweep* indipendente per ciascun membro, bensì come un'allocazione incrementale a livello di comunità basata su una *greedy algorithm*. L'obiettivo è massimizzare l'NPV della CER (VAN comune) distribuendo capacità di accumulo tra un sottoinsieme di membri candidati.

7.7.1 Problema di allocazione e ipotesi

Si consideri un insieme di membri abilitati $\mathcal{M}$ (presenti nella valutazione economica) e un sottoinsieme di candidati $\mathcal{C} \subseteq \mathcal{M}$ sui quali è consentito dimensionare l'accumulo. L'allocazione è descritta da un vettore di capacità aggregate $\mathbf{C} = \{C_i\}_{i \in \mathcal{C}}$, con $C_i \geq 0$ (kWh). Il modello energetico–economico definisce una funzione obiettivo:

$$\mathrm{NPV}_{\mathrm{CER}}(\mathbf{C}), \tag{7.37}$$

ottenuta eseguendo (i) il bilanciamento energetico con BESS per i membri interessati e ricomputando le grandezze della comunità (in particolare E_{cond}), e (ii) la valutazione economica con assunzioni tariffarie, fiscali e di investimento congelate, variando solo la taglia del BESS.

In pratica si introduce una quantizzazione della capacità tramite un passo fisso ΔC (kWh) e, opzionalmente, un vincolo sul totale allocabile $C_{\max}$.

7.7.2 Algoritmo greedy a incremento di capacità

L'algoritmo parte da una configurazione base $\mathbf{C}^{(0)} = \mathbf{0}$ e itera aggiungendo un incremento ΔC al singolo membro che massimizza l'incremento marginale di NPV dato lo stato corrente. Alla generica iterazione t:

1. Per ciascun candidato $i \in \mathcal{C}$ si costruisce un trial $\mathbf{C}^{(t)} + \Delta C\, \mathbf{e}_i$.
2. Per il trial si ricalcolano i flussi orari del membro i con il modello BESS (Capitolo 6, Sezione 6.6); tali serie sostituiscono quelle del run energetico corrente e vengono ricalcolate le grandezze aggregate della CER.
3. Si riesegue la valutazione economica mantenendo invariate tutte le assunzioni, eccetto la capacità iniziale del BESS del membro i (CAPEX/OPEX coerenti con la nuova taglia).
4. Si stima l'incremento marginale:

$$\Delta\mathrm{NPV}^{(t)}_{\mathrm{CER},i} = \mathrm{NPV}_{\mathrm{CER}}\big(\mathbf{C}^{(t)} + \Delta C\, \mathbf{e}_i\big) - \mathrm{NPV}_{\mathrm{CER}}\big(\mathbf{C}^{(t)}\big). \tag{7.38}$$

Si seleziona quindi

$$i^\star = \arg\max_{i \in \mathcal{C}} \Delta\mathrm{NPV}^{(t)}_{\mathrm{CER},i}, \tag{7.39}$$

e si accetta l'allocazione $\mathbf{C}^{(t+1)} = \mathbf{C}^{(t)} + \Delta C\, \mathbf{e}_{i^\star}$. In questo modo l'ottimizzazione tiene conto di interazioni e saturazioni: ogni trial è valutato rispetto al run "corrente" che incorpora già gli incrementi accettati nelle iterazioni precedenti.

7.7.3 Criteri di arresto e output

L'algoritmo si arresta quando si verifica almeno una delle seguenti condizioni: (i) superamento del limite totale $C_{\max}$; (ii) incremento marginale non positivo $\Delta\mathrm{NPV}^{(t)}_{\mathrm{CER},i^\star} \leq 0$ (se abilitato lo stop su marginalità negativa); (iii) assenza di candidati ammissibili.

Gli output sono salvati in una cartella `opt_{timestamp}` e includono: (a) una traccia iterativa dell'allocazione (step, membro selezionato, ΔNPV e taglie prima/dopo); (b) un riepilogo per membro con la capacità finale $C_i^\star$ e il contributo cumulato all'NPV; (c) una curva stepwise compatibile con la visualizzazione in dashboard; (d) metadati con parametri di run (ΔC, $C_{\max}$, candidati, run energetico di partenza). L'algoritmo è implementato nel modulo `cer_core/optimization/bess_greedy.py` ed è orchestrato dalla pagina Streamlit di valutazione economica.

7.7.4 Considerazioni

La procedura greedy è una euristica: non garantisce l'ottimo globale di $\mathrm{NPV}_{\mathrm{CER}}(\mathbf{C})$, ma fornisce una soluzione computazionalmente gestibile e interpretabile (allocazione "a pacchetti"), particolarmente adatta a scenari con molti membri e con funzione obiettivo non separabile per via dell'energia condivisa E_{cond}.

Capitolo 8

Caso studio: costruzione del dataset e impostazione dello scenario

Questo capitolo applica il simulatore sviluppato nei capitoli precedenti a un caso studio reale localizzato nel Comune di Doberdò del Lago (Provincia di Gorizia, Nord-Est Italia). L'obiettivo è duplice:

- costruire un dataset *coerente da capo a fondo* (consumi e produzione) a partire da informazioni raccolte sul campo e da fonti pubbliche, riallineandolo al formato richiesto dal simulatore (risoluzione temporale, anno base, strutture dati e convenzioni temporali);
- validare *qualitativamente* i profili di consumo sintetizzati dal modello bottom-up (Capitolo 5) tramite un confronto con profili medi provinciali pubblicati da ARERA/Portale Consumi (prelievi orari medi per mese e tipologia di giorno).

L'anno base scelto per le elaborazioni energetiche del caso studio è il 2023, sia per i consumi sia per la produzione fotovoltaica stimata, in modo da mantenere coerenza temporale con le convenzioni di sessione (Capitolo 3) e rendere direttamente confrontabili i risultati a valle (bilanciamento e indicatori economici). Il dataset ARERA utilizzato per il benchmark di forma (profilo medio provinciale) è invece riferito all'anno più recente disponibile al momento dell'analisi (2024); tale informazione è impiegata esclusivamente come *shape* di riferimento e viene poi riallineata su calendario e risoluzioni dell'anno base.

8.1 Raccolta dati primari: interviste in presenza e rilievo impianti

La prima fase del lavoro ha riguardato la raccolta di dati primari tramite interviste in presenza presso quattro utenze domestiche (di seguito *Utente 1–4*, anonimizzate). La raccolta in loco ha permesso di acquisire due famiglie di informazioni:

- **Consumi elettrici:** consumo annuo (kWh/anno) e informazioni sulle abitudini di utilizzo (orari di occupazione, utilizzo degli elettrodomestici, eventuali usi energivori), necessarie per parametrizzare il modello di domanda bottom-up (Capitolo 5).
- **Dotazioni impiantistiche (ove presenti):** potenza FV installata (kWp) e configurazione reale dell'impianto (orientamento/inclinazione e vincoli di installazione osservati in sede di sopralluogo). È stata inoltre verificata la presenza di accumulo elettrochimico (BESS) e la relativa capacità nominale (kWh), laddove disponibile, ai fini della successiva fase di bilanciamento (Capitolo 6).

Nel campione analizzato risultano due utenze *prosumer* (dotate di generazione FV) e due utenze *consumer* (solo consumo). Le principali caratteristiche raccolte sono riassunte in Tabella 8.1.

Tabella 8.1: Caratteristiche principali delle utenze intervistate (anno base 2023).

Utente	Ruolo	Consumo annuo [kWh]	FV installato [kWp]	BESS [kWh]	Classe potenza (ARERA)
Utente 1	Consumer	2000	–	–	3–4,5 kW
Utente 2	Prosumer	6000	15,5	12	> 6 kW
Utente 3	Prosumer	4500	8,25	4	> 6 kW
Utente 4	Consumer	2700	–	–	3–4,5 kW

8.2 Stima della produzione fotovoltaica: PVGIS e configurazione reale dei tetti

Per gli utenti prosumer (Utente 2 e Utente 3) non erano disponibili serie storiche complete e omogenee di produzione misurata. Coerentemente con l'approccio adottato nel modulo di produzione (Capitolo 4), la produzione FV è stata quindi stimata tramite PVGIS (JRC – Commissione Europea), ottenendo serie temporali orarie di output FV e variabili meteorologiche associate.

Per ciascun impianto sono stati impostati:

- **localizzazione:** Doberdò del Lago;
- **anno:** 2023 (anno base del caso studio);
- **potenza nominale:** pari alla potenza installata rilevata (Tabella 8.1);
- **parametri geometrici:** orientamento e inclinazione coerenti con la posa reale osservata sul tetto;
- **assunzioni di sistema:** perdite e ipotesi operative secondo le convenzioni del modello (Capitolo 4).

Nel caso studio non sono presenti impianti eolici; pertanto l'intero contributo di generazione rinnovabile locale è attribuito al fotovoltaico. Le serie orarie PVGIS sono state successivamente riallineate alle convenzioni temporali della sessione (Capitolo 3) e, dove necessario, portate alla risoluzione T_{15} del bilanciamento (Capitolo 6), preservando la coerenza energetica per intervallo.

8.3 Curve di consumo ARERA come benchmark: motivazione e fonte

Il profilo di domanda nel simulatore è generato tramite un modello bottom-up (Capitolo 5), quindi parametrico e dipendente dalle informazioni raccolte sul campo. Per ridurre il rischio di introdurre profili non realistici dal punto di vista della forma (distribuzione intra-giornaliera e stagionale), si è introdotta una validazione esterna: confrontare i profili sintetizzati dal simulatore con curve derivate da una fonte istituzionale *terza*.

A tale scopo sono stati utilizzati i dati pubblicati da ARERA/Portale Consumi relativi ai consumi medi di energia elettrica delle utenze domestiche[31], filtrabili per:

- Regione/Provincia,
- anno,
- classe di potenza,
- tipologia di mercato,
- residenza.

Tra le grandezze disponibili, il Portale rende accessibili profili di **prelievo medio orario** (in kWh) per i clienti domestici trattati orari, aggregati per provincia. Coerentemente con l'ambientazione del caso studio, sono stati scaricati i dati relativi alla Provincia di Gorizia e alle classi di potenza corrispondenti alle utenze intervistate (3–4,5 kW e >6 kW). Come anticipato in apertura, il dataset ARERA utilizzato per il benchmark è riferito al 2024: in questa applicazione viene impiegato come riferimento di forma e successivamente riallineato sul calendario dell'anno base.

8.4 Dal dato ARERA *mese* × *tipo giorno* a una serie annuale compatibile col simulatore

I file ARERA/Portale Consumi forniscono tipicamente un profilo medio orario distinto per:

- mese,
- tipologia di giorno (*giorno feriale*, *sabato*, *domenica*),
- ora (`Ora1` ... `Ora24`).

Per confrontare direttamente tali profili con quelli prodotti dal simulatore (serie continua su griglia temporale regolare), è stata implementata una procedura di ricostruzione che trasforma l'informazione ARERA in una serie annuale oraria e, se necessario, la converte alla risoluzione T_{15}.

8.4.1 Pulizia e ristrutturazione dei file (filter + pivot)

La prima fase consiste in:

1. conversione dei file scaricati (eventualmente in formato `xlsx`) in formato tabellare `csv`;
2. filtraggio per **Provincia** = **Gorizia** e per i campi anagrafici coerenti con il campione (ad es. residenza, ove disponibile);
3. normalizzazione dei nomi colonna e ristrutturazione in forma *tidy*, ottenendo una tabella indicizzata da (*mese*, *tipo_giorno*) con 24 colonne orarie (`Ora1` ... `Ora24`) contenenti il prelievo medio orario provinciale (kWh).

8.4.2 Costruzione del calendario annuale e mapping del *tipo giorno*

Si costruisce un calendario orario per l'intero anno base (2023), ottenendo una griglia di 8760 timestamp in convenzione coerente con la sessione (Capitolo 3). Per ciascun timestamp h (ora) si calcolano:

- mese $m(h) \in \{1, \ldots, 12\}$,
- ora del giorno $k(h) \in \{0, \ldots, 23\}$,
- giorno della settimana e relativa classificazione in:
  - lun–ven $\rightarrow$ *feriale*,
  - sab $\rightarrow$ *sabato*,
  - dom $\rightarrow$ *domenica*.

Questa classificazione consente di associare a ogni ora dell'anno la riga ARERA corretta (*mese* $\times$ *tipo_giorno*) e la colonna oraria corrispondente.

8.4.3 Join e ricostruzione della serie oraria

Il calendario viene unito (join) ai profili ARERA opportunamente portati in forma *long* $(m, \textit{tipo_giorno}, k) \mapsto$ kWh, ottenendo una serie oraria continua:

$$E^{T_H}_{\text{ARERA}}(h), \qquad \forall h \in \mathcal{H}_{2023}, \tag{8.1}$$

dove $\mathcal{H}_{2023}$ indica l'insieme delle ore dell'anno base e $E^{T_H}_{\text{ARERA}}(h)$ è l'energia media provinciale prelevata nell'ora h (kWh).

Si osserva che questa serie rappresenta una ricostruzione *in media provinciale* (non una misura puntuale della singola utenza); per tale ragione viene utilizzata come benchmark di *forma* e non come ground-truth quantitativo del singolo cliente.

8.4.4 Riscalatura al consumo annuo dell'utenza: da profilo "shape" a profilo "utente"

Per rendere confrontabile il benchmark ARERA con ciascuna utenza intervistata, la serie oraria viene riscalata imponendo che l'energia annua coincida con il consumo annuo osservato E_{utente}:

$$E^{T_H}_{\text{ARERA,scal}}(h) = E^{T_H}_{\text{ARERA}}(h) \, \frac{E_{\text{utente}}}{\sum\limits_{h \in \mathcal{H}_{2023}} E^{T_H}_{\text{ARERA}}(h)}. \tag{8.2}$$

In questo modo, il confronto tra profilo simulato e benchmark ARERA riguarda primariamente la *distribuzione temporale* dell'energia (intra-giornaliera e stagionale), mantenendo coerente l'energia totale annua del singolo utente.

8.4.5 Conversione $T_H \to T_{15}$ per confronto diretto

Il simulatore produce profili di domanda a 15 minuti (T_{15}). Per abilitare un confronto diretto sulla stessa griglia temporale, la serie oraria riscalata viene convertita a 15 minuti mediante una discretizzazione *piecewise-constant* che preserva l'energia:

$$E^{T_{15}}_{\text{ARERA,scal}}(t) = \frac{1}{4}\, E^{T_H}_{\text{ARERA,scal}}(h), \qquad \forall t \in h, \tag{8.3}$$

dove t indica un intervallo a 15 minuti e $t \in h$ significa che l'intervallo t appartiene all'ora h.

Questa scelta rende:

- il profilo ARERA e quello simulato confrontabili in T_{15};
- possibile effettuare confronti anche dopo aggregazione a orario (somma dei quarti d'ora) o su finestre temporali specifiche (ad es. mese, tipologia di giorno, fasce orarie).

8.5 Output atteso della fase dati e integrazione nello scenario

Al termine delle fasi descritte, il caso studio dispone dei seguenti artefatti, allineati temporalmente e pronti per l'esecuzione del bilanciamento energetico e del calcolo dell'energia condivisa (Capitolo 6):

- **serie di consumo per ciascun utente** generate dal modello bottom-up del simulatore (risoluzione T_{15}, anno base 2023), parametrizzate sui dati raccolti in intervista;
- **serie di produzione FV** per gli utenti prosumer (PVGIS, anno base 2023), riallineate alle convenzioni della sessione e rese compatibili con le strutture dati del simulatore;
- **serie "benchmark" di consumo** derivate da ARERA/Portale Consumi (Provincia di Gorizia, classe di potenza coerente), ricostruite come serie annuale,

riscalate al consumo annuo dell'utente e convertite alla risoluzione T_{15} per confronto qualitativo.

Questa fase costituisce quindi il punto di arrivo della costruzione del dataset: tutte le serie necessarie sono disponibili in forma tabellare, coerenti per anno base e risoluzione, e possono essere persistite nel perimetro di scenario secondo le convenzioni di audit e riproducibilità introdotte nei Capitoli 2–3.

8.6 Risultati del bilanciamento energetico: confronto tra profili ARERA e profili simulati

Una volta costruite e allineate temporalmente le serie di consumo e produzione (Sezioni 8.1–8.5), il passo successivo consiste nell'esecuzione del bilanciamento energetico descritto nel Capitolo 6, con calcolo delle grandezze a livello di singolo membro e di comunità: autoconsumi, prelievi e immissioni, nonché energia condivisa su base oraria E_{cond}.

Per isolare l'effetto dovuto alla *forma* del profilo di domanda, sono stati confrontati due run identici per configurazione di comunità e produzione rinnovabile (FV da PVGIS), differenziati unicamente dalla sorgente dei consumi:

- **Run ARERA (benchmark)**: domanda costruita a partire dai profili medi provinciali ARERA (Provincia di Gorizia, classe di potenza coerente), ricostruiti in serie annuale, riscalati sull'energia annua dell'utente e convertiti a T_{15} (Sezione 8.4).
- **Run simulato (bottom-up)**: domanda generata dal modello bottom-up del simulatore (Capitolo 5), parametrizzata sulle informazioni raccolte in intervista e riscalata all'energia annua osservata.

In entrambi i casi sono state mantenute le stesse dotazioni impiantistiche rilevate (Tabella 8.1), inclusi gli accumuli BESS laddove presenti (12 kWh per Utente 2 e 4 kWh per Utente 3), così da confrontare i risultati a parità di asset e variare esclusivamente la distribuzione temporale dei carichi.

8.6.1 KPI di comunità

La Tabella 8.2 riassume i KPI energetici principali a livello di CER ottenuti dai due run. In entrambi i casi l'energia annua complessiva prodotta e consumata rimane

Tabella 8.2: Confronto KPI di comunità (run ARERA vs run simulato).

KPI (CER)	**ARERA**	**Simulato**	**Δ (Sim–ARERA)**
$E_{\text{prod,tot}}$ [kWh]	26329,9	26329,9	0,0
$E_{\text{load,tot}}$ [kWh]	15200,0	15200,7	+0,7
$E_{\text{cond,tot}}$ [kWh]	1136,3	1469,7	+333,4
$E_{\text{import,tot}}$ [kWh]	6663,9	5636,9	-1026,9
$E_{\text{export,tot}}$ [kWh]	17400,8	16422,6	-978,2
`share_on_production`	0,0432	0,0558	+0,0126
`coverage_on_load`	0,0748	0,0967	+0,0219

sostanzialmente invariata (per costruzione del dataset e riscalatura), mentre variano le grandezze dipendenti dall'allineamento temporale tra domanda e produzione.

Il run con consumi simulati produce un'energia condivisa annua maggiore (circa +333 kWh, pari a $\sim +29\%$ rispetto al benchmark), e di conseguenza mostra:

- un aumento della quota di produzione che viene condivisa in comunità (`share_on_production`: 0,0432 → 0,0558);
- un aumento della copertura del carico tramite energia condivisa (`coverage_on_load`: 0,0748 → 0,0967);
- una riduzione sia dell'import totale da rete (-1027 kWh) sia dell'export totale (-978 kWh), coerente con un maggior impiego locale della generazione FV.

8.6.2 KPI per membro e impatto sui prosumer

L'effetto del diverso profilo di domanda si concentra sui membri prosumer, in particolare su Utente 2 che presenta la maggiore potenza FV installata. La Tabella 8.3 riporta le grandezze principali per i due prosumer (gli utenti consumer non dispongono di produzione locale e presentano quindi autoconsumo nullo e self-sufficiency pari a zero).

Il risultato più evidente riguarda la **self-sufficiency** di Utente 2, che passa da 0,799 (benchmark ARERA) a 0,862 (profilo simulato). Tale incremento è coerente con l'aumento dell'autoconsumo annuo (+380 kWh circa) e con la contestuale riduzione del prelievo da rete (~ -380 kWh), a parità di produzione FV annua. Un comportamento analogo, sebbene di entità leggermente inferiore in termini assoluti, è osservabile anche per Utente 3.

Tabella 8.3: Confronto KPI per i membri prosumer (run ARERA vs run simulato).

Membro	KPI	ARERA	Simulato
Utente 2	$E_{\text{aut,tot}}$ [kWh]	4794,6	5174,7
	$E_{\text{imm,tot}}$ [kWh]	11449,4	11128,6
	$E_{\text{prel,tot}}$ [kWh]	1205,4	825,7
	`self_consumption`	0,290	0,313
	`self_sufficiency`	0,799	0,862
Utente 3	$E_{\text{aut,tot}}$ [kWh]	2605,2	2919,4
	$E_{\text{imm,tot}}$ [kWh]	7087,7	6763,7
	$E_{\text{prel,tot}}$ [kWh]	1894,8	1580,8
	`self_consumption`	0,266	0,298
	`self_sufficiency`	0,579	0,649

8.6.3 Discussione: interpretazione delle differenze e bias "diurno" del profilo simulato

Nel complesso, i due run producono risultati *in linea* e le differenze quantitative restano contenute rispetto alle energie annue in gioco. Ciò suggerisce che il dataset costruito a partire da ARERA fornisce un benchmark coerente, mentre il modello bottom-up genera profili plausibili dal punto di vista energetico globale.

La differenza sistematica osservata (maggior energia condivisa e maggior self-sufficiency nel run simulato) è riconducibile alla **maggiore correlazione temporale** tra carico e generazione FV introdotta dal modello di domanda. In particolare, la parametrizzazione basata su interviste ha evidenziato la tendenza dei consumatori a concentrare l'uso dei carichi più impegnativi (ad es. lavatrice, lavastoviglie, pompa di calore) nelle **ore di luce**, anche con l'obiettivo implicito di massimizzare l'uso dell'energia prodotta localmente. Questo effetto emerge soprattutto per Utente 2: essendo il membro con l'impianto FV di taglia maggiore, lo spostamento di carico verso le ore diurne produce un incremento più marcato dell'autoconsumo e una riduzione del prelievo da rete, migliorando di conseguenza la self-sufficiency (0,79 $\rightarrow$ 0,86).

Dal punto di vista metodologico, tale risultato è utile perché:

- conferma che la *forma* del profilo di domanda (non solo l'energia annua) è determinante per E_{cond} e per gli indicatori di autoconsumo/autosufficienza;
- evidenzia un possibile *bias ottimistico* del profilo simulato quando le abitudini dichiarate tendono a sovrastimare la capacità di spostare carichi (flessibilità comportamentale).

In prospettiva, una calibrazione del modello bottom-up su misure reali (smart meter) consentirebbe di quantificare tale bias e raffinare la stima della flessibilità effettivamente disponibile, migliorando l'affidabilità delle stime di energia condivisa e dei KPI economici a valle.

8.7 Valutazione economico-finanziaria: assunzioni di prezzo, incentivi e asset

La valorizzazione economica dei flussi energetici ottenuti dal bilanciamento (Sezione 8.6) è eseguita tramite il modulo di valutazione descritto nel Capitolo 7. In questa fase i profili energetici orari (prelievi, immissioni e energia condivisa) vengono trasformati in grandezze economiche annue (costi, ricavi e incentivi) e successivamente proiettati su un orizzonte DCF.

8.7.1 Prezzi di acquisto e vendita

Per la componente di **acquisto energia** si è adottata una tariffazione a fasce (**F1, F2, F3**) differenziata per utenza, con l'obiettivo di utilizzare prezzi unitari *realistici* e coerenti con profili di consumo domestici. I prezzi sono stati ricavati tramite il Portale Offerte[32]: per ciascun utente sono stati inseriti i dati di consumo e, a partire dai valori economici stimati per un insieme di configurazioni/offerte, è stato calcolato un **prezzo unitario equivalente** in €/kWh (per fascia) come media dei valori ottenuti. In questo modo la valorizzazione del prelievo residuo da rete (`cost_buy`) riflette l'ordine di grandezza e la struttura a fasce tipica delle bollette domestiche.

Per la componente di **vendita energia** si è assunto come prezzo di cessione il **PZO della macroarea NORD**. Il profilo PZO è stato acquisito tramite il Servizio API del GME[33]. Per vincoli di licenza e di utilizzo associati al Servizio API, i dati grezzi scaricati non vengono redistribuiti nel presente elaborato[34].

Nel DCF, i **prezzi unitari** di acquisto e vendita sono proiettati su 20 anni applicando un tasso di inflazione annuo del **2%**.

8.7.2 Incentivi: TIP e TIAD

Gli incentivi sono calcolati sull'energia condivisa su base oraria $E_{\text{cond}}(h)$, coerentemente con la definizione e gli output del modulo di bilanciamento (Capitolo 6). Nel caso studio:

- la **TIP** (`rev_tip`) è calcolata secondo l'impostazione implementata nel simulatore (`rse_decree`);
- la **TIAD** (`rev_tiad`) è calcolata con parametrizzazione coerente con una CER in macroarea NORD (implementazione `rse_arera`).

Per quanto riguarda la **potenza incentivante**, entrambi gli impianti FV dei prosumer sono stati considerati interamente ammissibili (nessuna riduzione della potenza incentivabile). Inoltre non sono state considerate riduzioni in conto capitale (nessun contributo a fondo perduto o conto impianti). In assenza di indicazioni specifiche su scenari regolatori futuri, gli incentivi sono mantenuti *costanti* lungo l'orizzonte di analisi.

8.7.3 Asset e ipotesi di CAPEX/OPEX

In questa versione del caso studio gli asset sono valorizzati includendo un investimento iniziale (**CAPEX**) e costi operativi annui (**OPEX**). I valori unitari adottati, discussi con RSE, sono:

- **Fotovoltaico (PV):** CAPEX = 1100 €/kW, OPEX = 18 €/kW/anno;
- **Batteria (BESS):** CAPEX = 600 €/kWh, OPEX = 0,01 €/kWh/anno.

Per ciascun prosumer i, il CAPEX iniziale è calcolato come:

$$\mathrm{CAPEX}_i = 1100 \cdot P_{\mathrm{PV},i} \; + \; 600 \cdot C_{\mathrm{BESS},i},$$

dove $P_{\mathrm{PV},i}$ è la potenza FV installata (kW) e $C_{\mathrm{BESS},i}$ la capacità nominale dell'accumulo (kWh), coerenti con i dati di Tabella 8.1. Analogamente, l'OPEX annuo è:

$$\mathrm{OPEX}_i = 18 \cdot P_{\mathrm{PV},i} \; + \; 0{,}01 \cdot C_{\mathrm{BESS},i}.$$

8.7.4 Finanza e fiscalità

L'orizzonte DCF è pari a 20 anni, con tasso di sconto $r = 3\%$. Nella proiezione temporale sono state applicate le seguenti ipotesi di inflazione:

- **prezzi di acquisto energia:** +2% annuo;
- **prezzi di vendita energia:** +2% annuo;
- **OPEX:** +2% annuo.

È inoltre considerata un'**aliquota di detrazione del 50%** sul CAPEX eleggibile (assunzione coerente con il quadro di incentivi attualmente disponibile al momento dell'analisi). Coerentemente con la parametrizzazione descritta in Sezione 7.2.7, la detrazione non viene trattata come riduzione del CAPEX netto iniziale; al contrario, è modellata come **beneficio di cassa addizionale** ripartito in quote annuali $D_i(y)$ nei primi N_{det} anni dell'orizzonte. In altre parole, il CAPEX al tempo zero rimane invariato e la detrazione entra nei flussi di cassa come incremento dei cash flow negli anni successivi. Nel set di simulazioni qui riportato, il contributo risulta posto a zero nei file di export (`tax_detraction`=0), per cui i KPI riportati sono conservativi rispetto a uno scenario con detrazione attiva. La tassazione ordinaria è altrimenti disattivata (nessuna imposta sul reddito/utile modellata nel DCF).

8.8 Risultati economico-finanziari: confronto tra profili simulati e benchmark ARERA

Questa sezione confronta gli output economico-finanziari ottenuti con (i) profili di consumo *simulati* (modello bottom-up, Capitolo 5) e (ii) profili *ARERA* riscalati (Sezione 8.4), mantenendo **invariati** asset, ipotesi di prezzo e meccanismi incentivanti (Sezione 8.7). I valori riportati sono calcolati a partire dai file di export del simulatore (cash flow: `Cash flow totale_dati_simulati.csv` e `Cash flow totale_dati_ARERA.csv`; conto economico: `PnL totale_dati_simulati.csv` e `PnL totale_dati_ARERA.csv`).

8.8.1 KPI complessivi

La Tabella 8.4 sintetizza i principali KPI DCF a livello di comunità. I KPI sono quindi da intendersi *conservativi* rispetto a un eventuale scenario in cui la detrazione fiscale (Sezione 7.2.7) sia effettivamente contabilizzata come beneficio di cassa.

Tabella 8.4: Confronto KPI economico-finanziari complessivi (run ARERA vs run simulato).

KPI (totale CER)	**ARERA**	**Simulato**	**Δ (Sim−ARERA)**
NPV / VAN [€]	35 875,861	39 334,589	+3 458,728
IRR / TIR [–]	0,080	0,086	+0,006
Payback [anni]	11	10	-1
Discounted Payback [anni]	11	10	-1

Il run con dati simulati presenta un miglioramento sistematico dei KPI: l'incremento di VAN è pari a +3 458,728 €(circa +9,6% rispetto al benchmark), mentre la TIR passa da 8,0% a 8,6%. Coerentemente, il tempo di rientro si riduce di circa un anno (Payback e Discounted Payback). Il risultato è coerente con quanto discusso nella Sezione 8.6: a fronte della stessa produzione FV, i profili simulati aumentano l'energia condivisa e l'autoconsumo (Tabella 8.2 e Tabella 8.3), migliorando la valorizzazione economica complessiva.

8.8.2 Conto economico (PnL): principali driver

La variazione dei KPI economici può essere letta tramite le principali voci del conto economico annuale: ricavi da vendita (rev_sell), ricavi da incentivazione (rev_tip e rev_tiad) e costo di acquisto da rete (cost_buy). La Tabella 8.5 riporta un confronto sull'anno base (Anno 1).

Tabella 8.5: Confronto conto economico (Anno 1): run ARERA vs run simulato.

Voce (Anno 1)	**ARERA [€]**	**Simulato [€]**	**Δ [€]**
rev_sell	1 762,685	1 698,023	-64,663
rev_tip	146,421	189,814	+43,392
rev_tiad	13,522	17,489	+3,967
cost_buy	2 020,742	1 856,598	-164,144
EBITDA	-621,614	-474,773	+146,841

Si osserva che, nel run simulato, aumentano i ricavi da incentivazione (rev_tip e rev_tiad), coerentemente con la maggiore energia condivisa, mentre si riducono sia il costo di acquisto da rete (cost_buy) sia i ricavi da vendita (rev_sell). Quest'ultimo effetto è interpretabile come conseguenza diretta della minore energia esportata: una quota maggiore di produzione viene infatti valorizzata internamente tramite autoconsumo/condivisione.

A livello cumulato sull'orizzonte di analisi (somma anni 1–20), la dinamica è confermata: i maggiori ricavi da incentivazione e la riduzione di cost_buy più che compensano la riduzione di rev_sell, migliorando l'EBITDA complessivo. Per completezza, la Tabella 8.6 riporta le somme delle principali voci (anni 1–20).

8.8.3 Flussi di cassa e decomposizione del ΔVAN

Il cash flow annuo della comunità (anni 1–20) è costruito, coerentemente con il Capitolo 7, come somma di: risparmio da autoconsumo (`savings_autocons`), ricavi

Tabella 8.6: Somma delle principali voci di conto economico (anni 1–20): run ARERA vs run simulato.

Voce (anni 1–20)	**ARERA [€]**	**Simulato [€]**	**Δ [€]**
rev_sell	42 828,619	41 257,483	-1 571,136
rev_tip	2 928,421	3 796,270	+867,849
rev_tiad	270,441	349,789	+79,348
cost_buy	49 098,716	45 110,451	-3 988,265
opex	12 719,673	12 719,673	0,000
EBITDA	-15 790,908	-12 426,581	+3 364,326

da vendita (rev_sell), ricavi da incentivazione (rev_tip e rev_tiad), OPEX e (eventuali) sostituzioni (`capex_replacement`). Nel dataset esportato, l'unica sostituzione modellata è pari a 2,850 €al Year 12, identica in entrambi i run.

La differenza di VAN tra simulato e ARERA (+3 458,728 €) è pienamente spiegabile scomponendo i contributi attualizzati delle componenti di cash flow. La Tabella 8.7 riassume tale decomposizione.

Tabella 8.7: Decomposizione del ΔVAN (attualizzato) tra run simulato e run ARERA.

Componente	**Contributo a ΔVAN [€]**
Maggior risparmio da autoconsumo (`savings_autocons`)	+4 046,196
Maggiori ricavi da incentivazione (TIP+TIAD)	+899,241
Minori ricavi da vendita (rev_sell)	-1 486,709
Totale	+3 458,728

Il driver principale risulta quindi l'aumento del risparmio da autoconsumo, seguito dall'incremento dei ricavi da incentivazione, mentre la riduzione dei ricavi da vendita compensa parzialmente il beneficio complessivo. Questo spiega anche l'anticipo del punto di pareggio: nel run ARERA il cumulato attualizzato rimane negativo al Year 10 (-1 178,557 €) e diventa positivo al Year 11 (2 548,692 €), mentre nel run simulato è già positivo al Year 10 (467,393 €).

8.8.4 Confronto con simulatori esistenti e chiusura del capitolo

Il Payback pari a 10 anni ottenuto nel run con dati simulati è in linea con gli ordini di grandezza tipicamente restituiti da strumenti di simulazione disponibili per configurazioni di condivisione energetica, come ad esempio il tool *CACER_Simulator* sviluppato da RSE[30]. Nel complesso, il caso studio conferma che la **rappresentazione realistica dei profili orari di domanda** ha un impatto diretto sulla

fattibilità economica, in quanto modifica l'energia condivisa e l'autoconsumo e quindi (i) i ricavi incentivanti e (ii) i risparmi da mancato acquisto di energia da rete.

Capitolo 9

Conclusioni e sviluppi futuri

Questo lavoro ha presentato la progettazione e l'implementazione di un simulatore modulare end-to-end per l'analisi di Comunità Energetiche Rinnovabili (CER), in grado di ricostruire (i) i flussi energetici dei singoli membri e della comunità e (ii) i corrispondenti risultati economico-finanziari, garantendo riproducibilità e tracciabilità tramite persistenza su filesystem e versionamento dei run. A valle del caso studio e dei confronti effettuati, questa sezione riassume i principali margini di estensione del simulatore e chiude con una sintesi del contributo fornito.

9.1 Sviluppi futuri del simulatore

Gli sviluppi futuri più interessanti, in continuità con l'architettura e con i moduli già implementati, possono essere organizzati in tre linee principali: (i) estensione della componente di produzione e della sua rappresentazione geometrica, (ii) arricchimento del modello di domanda con nuovi carichi e nuovi comportamenti, (iii) introduzione di procedure di ottimizzazione più ampie che includano non solo l'accumulo ma anche gli asset di produzione.

9.1.1 Produzione: acquisizione dati e generazione automatica di render 3D tramite reti convoluzionali

Nel simulatore attuale, la definizione delle aree fotovoltaiche e la generazione dei render 3D del tetto e dei pannelli supportano la validazione qualitativa della configurazione (packing, vincoli geometrici, spaziature), ma richiedono input espliciti in interfaccia e una costruzione manuale delle geometrie. Un'evoluzione naturale è automatizzare (parzialmente o totalmente) la ricostruzione 3D e/o l'estrazione delle

superfici installabili attraverso modelli basati su reti convoluzionali (CNN) e, più in generale, su computer vision.

L'obiettivo applicativo può essere formulato come: dato un insieme di osservazioni (ad esempio immagini del tetto, fotografie terrestri o viste satellitari), stimare in modo automatico una rappresentazione 3D e/o un insieme di poligoni "installabili" da usare come input del modulo FV. In termini pratici, la pipeline potrebbe includere:

- **segmentazione e classificazione delle superfici**: identificazione di falde, ostacoli (camini, abbaini), aree d'ombra permanenti e margini non installabili;
- **stima di geometria e orientamento**: inclinazione e azimut delle falde, eventualmente con incertezza associata;
- **ricostruzione 3D**: passaggio da immagini a mesh (o modello semplificato) coerente con il renderer già previsto;
- **derivazione di input canonici**: conversione del risultato in GeoJSON/poligoni e parametri geometrici compatibili con la logica di packing e con le chiamate a PVGIS.

La criticità principale di questa direzione non è tanto l'integrazione software (che si innesterebbe naturalmente nel ramo "Producers"), quanto la **disponibilità di dati per addestramento e validazione**. In un contesto reale, ottenere dataset supervisionati (immagini → etichette installabili e/o mesh 3D "ground truth") richiede un lavoro consistente: raccolta dati, annotazione, definizione di standard di etichettatura e gestione delle variabilità (materiali, condizioni meteo, qualità delle immagini, occlusioni). Un approccio pragmatico può prevedere una fase iniziale con:

- **dati sintetici**: generazione di tetti e scenari simulati per creare esempi etichettati automaticamente;
- **transfer learning**: riuso di modelli pre-addestrati per segmentazione/monodepth, adattati con un dataset ridotto ma mirato;
- **human-in-the-loop**: un workflow in cui il modello propone aree/mesh e l'utente corregge in UI, riducendo il costo di annotazione e mantenendo auditabilità.

Il valore aggiunto, lato simulatore, sarebbe duplice: (i) ridurre l'onere di inserimento manuale della geometria e (ii) aumentare la coerenza tra "asset reale" e configurazione simulata, rendendo più affidabili sia la potenza installabile sia la validazione visiva.

9.1.2 Consumatori: integrazione di nuovi carichi (es. veicoli elettrici) con approccio bottom-up

Il modello dei consumi è stato costruito con una logica **bottom-up**: il profilo totale deriva dalla somma di componenti semplici e interpretabili (baseload, occupazione, cucina, lavanderia, climatizzazione), ciascuna con parametri espliciti e output intermedi salvati. Questa scelta rende naturale estendere il simulatore introducendo **nuovi impianti/elettrodomestici** senza stravolgere l'architettura.

In particolare, l'integrazione dell'EV può essere realizzata in modo coerente con i moduli già presenti adottando una pipeline minimale:

- definizione (da dati reali o distribuzioni) di **orari di arrivo/partenza** e **chilometraggio giornaliero**;
- conversione km → kWh tramite efficienza del veicolo e definizione del **fabbisogno di ricarica**;
- generazione del profilo di potenza rispettando **potenza EVSE** e **vincoli di disponibilità**;
- opzionale **spostamento del ciclo di ricarica** verso ore di maggiore produzione FV o verso fasce a minor costo (F1–F3), mantenendo trasparente il criterio di decisione.

Questa impostazione "a blocchi" consente di aumentare il realismo della domanda mantenendo **controllo e auditabilità**: ogni nuovo carico è tracciabile e attivabile/disattivabile per analisi di sensibilità e scenari what-if.

9.1.3 Ottimizzazione: estensione dalla sola batteria all'ottimizzazione degli impianti FV ed eolici

Nella versione attuale, l'ottimizzazione è concentrata sul dimensionamento/allocazione del BESS in funzione dell'NPV, tramite un'euristica greedy che ricalcola bilanciamento ed economia per configurazioni candidate. Un'evoluzione naturale è estendere l'ottimizzazione includendo gli **asset di produzione** (fotovoltaico ed eolico), con obiettivi e vincoli compatibili con studi di fattibilità CER.

Esempi di problemi ottimizzabili includono:

- **sizing FV**: scelta della potenza installata (o del numero di moduli) e, dove applicabile, selezione delle aree effettivamente utilizzate;

- **orientamento/inclinazione**: ricerca di tilt/azimut ottimali (o discretizzati) per massimizzare autoconsumo, energia condivisa o NPV;
- **sizing eolico**: scelta del modello turbina e della taglia, con vincoli su quota mozzo, perdite e disponibilità del sito;
- **ottimizzazione integrata**: co-design di FV, eolico e BESS, tenendo conto delle interazioni tra profili di domanda, produzione e meccanismi di valorizzazione (energia condivisa, vendita, risparmio da autoconsumo).

Dal punto di vista metodologico, questa estensione richiede:

- una funzione obiettivo esplicita (NPV, payback, energia condivisa, copertura del carico, o formulazioni multi-obiettivo);
- vincoli realistici (superfici, limiti autorizzativi, budget, vincoli tecnici);
- strategie computazionali robuste (grid search, metaeuristiche, oppure ottimizzazione bayesiana) per gestire lo spazio di ricerca senza esplodere il numero di run.

Un punto di forza del simulatore è che la struttura "scenario → run energetico → run economico", con contratti dati stabili e output versionati, rende queste iterazioni intrinsecamente tracciabili e confrontabili: ogni soluzione candidata è un run ispezionabile, non una configurazione "opaca".

9.2 Conclusioni: utilità del simulatore e modularità per riuso e analisi specialistiche

Nel complesso, il simulatore sviluppato risulta utile come **strumento di comprensione e studio di fattibilità** delle comunità energetiche, perché consente di:

- quantificare e spiegare i flussi energetici chiave (prelievi, immissioni, autoconsumo, energia condivisa) su base temporale coerente con la regolazione;
- tradurre tali flussi in risultati economico-finanziari comparabili tra scenari, con assunzioni esplicite e salvate a corredo dei run;
- evidenziare i benefici potenziali della CER (risparmio da autoconsumo, incentivi su energia condivisa, ricavi da vendita) e la loro sensibilità rispetto agli input.

Queste caratteristiche sono particolarmente rilevanti in fase di progettazione, quando l'obiettivo non è soltanto stimare un valore finale, ma comprendere **perché** un certo asset o un certo comportamento (ad esempio lo spostamento dei carichi) modifica l'energia condivisa e, quindi, la convenienza economica.

Un ulteriore elemento conclusivo riguarda la **modularità** del simulatore. La separazione tra orchestrazione (UI) e logica di dominio (core), insieme alla persistenza degli artefatti e ai contratti dati tra moduli, consente di scomporre il sistema e riutilizzare sottoparti in contesti più specifici. Esempi concreti di riuso includono:

- isolare il sottosistema consumi per sviluppare analisi mirate su **scelte tariffarie** e simulazioni "bolletta vs profilo" (ad esempio confrontando strutture F1–F3 o offerte indicizzate);
- utilizzare il ramo produttori per stimare, in modo riproducibile, **producibilità FV/eolica** su località e configurazioni diverse, anche indipendentemente dalla parte CER;
- sostituire input (profili esterni) mantenendo invariati bilanciamento ed economia, grazie al ponte di integrazione e alla definizione di input canonici di scenario.

In sintesi, il lavoro dimostra che un simulatore progettato con attenzione a riproducibilità, tracciabilità e modularità non è solo un "calcolatore di KPI", ma un supporto operativo per esplorare scenari, motivare decisioni progettuali e comunicare in modo chiaro i benefici e i trade-off associati a una comunità energetica.

Bibliografia

[1] Parlamento Europeo e Consiglio, *Direttiva (UE) 2018/2001 del Parlamento Europeo e del Consiglio dell'11 dicembre 2018 sulla promozione dell'uso dell'energia da fonti rinnovabili (RED II)*, Gazzetta Ufficiale dell'Unione Europea, 2018.

[2] Parlamento Europeo e Consiglio, *Direttiva (UE) 2019/944 del Parlamento Europeo e del Consiglio del 5 giugno 2019 relativa a norme comuni per il mercato interno dell'energia elettrica*, Gazzetta Ufficiale dell'Unione Europea, 2019.

[3] «Testo Integrato Autoconsumo Diffuso (TIAD),» ARERA – Autorità di Regolazione per Energia Reti e Ambiente, 2024, Allegato A alla deliberazione 727/2022/R/eel, come integrato e modificato dalla deliberazione 15/2024/R/eel. indirizzo: `https://www.arera.it/fileadmin/allegati/docs/22/727-22TIAD.pdf`

[4] «Decreto CACER e TIAD – Regole operative per l'accesso al servizio per l'autoconsumo diffuso e al contributo PNRR (Allegato 1),» GSE – Gestore dei Servizi Energetici, 2025, Versione del 16 luglio 2025. indirizzo: `https://www.gse.it/documenti_site/Documenti%20GSE/Servizi%20per%20te/AUTOCONSUMO/Gruppi%20di%20autoconsumatori%20e%20comunita%20di%20energia%20rinnovabile/Regole%20e%20procedure/ALLEGATO%201%20Regole%20Operative%20CACER.pdf`

[5] Ministero dell'Ambiente e della Sicurezza Energetica, *Decreto del Ministero dell'Ambiente e della Sicurezza Energetica 7 dicembre 2023, n. 414 (Decreto CACER)*, 2023. indirizzo: `https://www.tb.camcom.gov.it/uploads/AMB/normative/Decreto%20MASE%20n.414%20del%2007.12.2023.pdf`

[6] Open-Meteo. «Open-Meteo: Free Weather API. »indirizzo: `https://open-meteo.com/`

[7] Joint Research Centre, European Commission. «PVGIS – Photovoltaic Geographical Information System. »indirizzo: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis_en

[8] European Commission, Joint Research Centre. «PVGIS API: Non-interactive service. »indirizzo: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/getting-started-pvgis/api-non-interactive-service_en

[9] European Commission, Joint Research Centre. «PVGIS tool: Hourly radiation / hourly PV output. »indirizzo: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/pvgis-tools/hourly-radiation_en

[10] J. F. Manwell, J. G. McGowan e A. L. Rogers, *Wind Energy Explained: Theory, Design and Application*, 2ª ed. John Wiley & Sons, 2009, ISBN: 9780470015001. indirizzo: https://www.wiley.com/en-gb/Wind%2BEnergy%2BExplained%3A%2BTheory%2C%2BDesign%2Band%2BApplication%2C%2B2nd%2BEdition-p-9780470015001

[11] J. Freeman, P. Gilman, J. Jorgenson e T. Ferguson, «Reference Manual for the System Advisor Model's Wind Power Performance Model,» National Renewable Energy Laboratory (NREL), rapp. tecn. NREL/TP-6A20-60570, 2014. DOI: 10.2172/1150800 indirizzo: https://docs.nrel.gov/docs/fy14osti/60570.pdf

[12] C. G. Justus e A. Mikhail, «Height variation of wind speed and wind distributions statistics,» *Geophysical Research Letters*, vol. 3, n. 5, pp. 261–264, 1976. DOI: 10.1029/GL003I005P00261

[13] International Electrotechnical Commission (IEC), *IEC 61400-12-1:2022 Wind energy generation systems – Part 12-1: Power performance measurements of electricity producing wind turbines*, IEC Webstore, 2022. indirizzo: https://webstore.iec.ch/en/publication/68499

[14] A. Capasso, W. Grattieri, R. Lamedica e A. Prudenzi, «A bottom-up approach to residential load modeling,» *IEEE Transactions on Power Systems*, vol. 9, n. 2, pp. 957–964, 1994. DOI: 10.1109/59.317650

[15] I. Richardson, M. Thomson e D. Infield, «Domestic electricity use: A high-resolution energy demand model,» *Energy and Buildings*, vol. 42, n. 10, pp. 1878–1887, 2010. DOI: 10.1016/j.enbuild.2010.05.023

[16] N. Pflugradt, P. Stenzel, L. Kotzur e D. Stolten, «LoadProfileGenerator: An Agent-Based Behavior Simulation for Generating Residential Load Profiles,» *Journal of Open Source Software*, vol. 7, n. 71, p. 3574, 2022. DOI: 10.21105/joss.03574

[17] G. Besagni, L. Premoli Vilà e M. Borgarello, «Italian Household Load Profiles: A Monitoring Campaign,» *Buildings*, vol. 10, n. 12, p. 217, 2020. DOI: 10.3390/buildings10120217

[18] G. Azzolini et al., «Aggiornamento delle metodologie per la stima dei consumi energetici nel settore residenziale e analisi dei dati del database delle detrazioni fiscali mediante la metodologia sviluppata,» ENEA – Agenzia nazionale per le nuove tecnologie, l'energia e lo sviluppo economico sostenibile, rapp. tecn. RdS/PAR 2017/086, 2017, Ministero dello Sviluppo Economico, Ricerca di Sistema.

[19] E. McKenna e M. Thomson, «High-resolution stochastic integrated thermal–electrical domestic demand model,» *Applied Energy*, vol. 165, pp. 445–461, 2016. DOI: 10.1016/j.apenergy.2015.12.032

[20] J. Widén e E. Wäckelgård, «A high-resolution stochastic model of domestic activity patterns and electricity demand,» *Applied Energy*, vol. 87, n. 6, pp. 1880–1892, 2010. DOI: 10.1016/j.apenergy.2009.11.006

[21] I. Richardson, M. Thomson, D. Infield e C. Clifford, «A high-resolution domestic building occupancy model for energy demand simulations,» *Energy and Buildings*, vol. 40, n. 8, pp. 1560–1566, 2008. DOI: 10.1016/j.enbuild.2008.02.006

[22] M. Stokes, M. Rylatt e K. Lomas, «A simple model of domestic lighting demand,» *Energy and Buildings*, vol. 36, n. 2, pp. 103–116, 2004. DOI: 10.1016/j.enbuild.2003.10.007

[23] I. Richardson, M. Thomson, D. Infield e A. Delahunty, «Domestic lighting: A high-resolution energy demand model,» *Energy and Buildings*, vol. 41, n. 7, pp. 781–789, 2009. DOI: 10.1016/j.enbuild.2009.02.010

[24] European Commission, *Commission Regulation (EU) 2019/2022 of 1 October 2019 laying down ecodesign requirements for household dishwashers pursuant to Directive 2009/125/EC*, Official Journal of the European Union, 2019. indirizzo: https://eur-lex.europa.eu/eli/reg/2019/2022/oj/eng

[25] European Commission, *Commission Delegated Regulation (EU) 2019/2017 of 11 March 2019 with regard to energy labelling of household dishwashers and repealing Delegated Regulation (EU) No 1059/2010*, Official Journal of the European Union, 2019. indirizzo: https://eur-lex.europa.eu/eli/reg_del/2019/2017/oj/eng

[26] European Commission, *Commission Regulation (EU) 2019/2023 of 1 October 2019 laying down ecodesign requirements for household washing machines and household washer-dryers pursuant to Directive 2009/125/EC*, Official Journal of the European Union, 2019. indirizzo: https://eur-lex.europa.eu/eli/reg/2019/2023/oj/eng

[27] European Commission, *Commission Delegated Regulation (EU) 2019/2014 of 11 March 2019 with regard to energy labelling of household washing machines and household washer-dryers and repealing Delegated Regulation (EU) No 1061/2010 and Directive 96/60/EC*, Official Journal of the European Union, 2019. indirizzo: https://eur-lex.europa.eu/eli/reg_del/2019/2014/oj/eng

[28] J. Vivian, A. Zarrella, G. Emmi e M. De Carli, «Analysis of simplified lumped-capacitance models to simulate the thermal behaviour of buildings,» in *Preprints of BSA 2017 – Building Simulation Applications*, 2017.

[29] H. Pieper, I. Krupenski, W. B. Markussen, T. S. Ommen, A. Siirde e A. Volkova, «Method of linear approximation of COP for heat pumps and chillers based on thermodynamic modelling and off-design operation,» *Energy*, vol. 230, p. 120 743, 2021. DOI: 10.1016/j.energy.2021.120743

[30] RSE-CoLabs, *CACER_Simulator: simulation tool for renewable energy sharing configurations*, 2025. indirizzo: https://github.com/RSE-CoLabs/CACER_Simulator

[31] ARERA – Autorità di Regolazione per Energia Reti e Ambiente. «Analisi dei consumi dei clienti domestici. »indirizzo: https://www.arera.it/dati-e-statistiche/dettaglio/analisi-dei-consumi-dei-clienti-domestici

[32] Acquirente Unico. «Il Portale Offerte: confronto tariffe e prezzi luce e gas. »indirizzo: https://www.ilportaleofferte.it/portaleOfferte/it/confronto-tariffe-prezzi-luce-gas.page

[33] Gestore dei Mercati Energetici (GME). «Interfaccia API. »indirizzo: https://www.mercatoelettrico.org/it-it/Home/InterfacciaAPI

[34] «Condizioni generali di utilizzo dei dati di mercato di proprietà del Gestore dei Mercati Energetici S.p.A. accessibili mediante servizio API,» Gestore dei Mercati Energetici (GME), 2025. indirizzo: https://www.mercatoelettrico.org/Portals/0/Documents/it-IT/20251015CONDIZIONI_GENERALI_API.pdf